

COMPUTER VISION PROJECT

Classical Object Detection and Localisation of the University of Ghana Coat of Arms

Template matching, sliding window search, feature matching and colour segmentation, evaluated on an annotated test set

Author	Nii Yartey Gidiglo
Repository	github.com/Qwamzz/Computer-Vision-Projects
Target object	University of Ghana coat of arms
Evaluation set	36 annotated scenes, 38 object instances
Implementation	Python 3 and OpenCV, no pre-trained recognition model

Statement on the imagery. Every image used in this project is a real photograph or a real vector emblem obtained from Wikimedia Commons under a free licence, and the licence and author of each file are listed in Section 2.2 and in data/SOURCES.md. No image was produced by an image generation model.

1. Problem definition and objectives

The task is to build a system that finds a chosen object category in an image and reports where it is, using classical detection techniques rather than a pre-trained recognition network. The object chosen here is the **University of Ghana coat of arms**, the emblem that carries three stylised palm trees over a navy shield above the motto *Integri Procedamus*. It is a good subject for a classical detector: it is rigid, it is planar, it has a fixed internal layout and it has strong internal contrast, which is exactly the situation in which correlation based matching is defensible. It is also a realistic object to detect, since it appears on university signage, letterheads, certificates, identity cards and merchandise, and automatic detection supports document verification and brand monitoring.

The specific objectives are the following.

- Build a template of the object and a test set in which the object appears at different positions, sizes, orientations, illumination levels, backgrounds and degrees of partial occlusion.
- Implement template matching by normalised cross correlation and measure how the decision threshold trades true detections against false and missed detections.
- Extend the detector with an explicit sliding window search and measure the effect of window size, step size and threshold on both accuracy and computational cost.
- Add feature matching with RANSAC and a colour segmentation stage where these improve reliability, and draw a bounding box on every detected instance.
- Evaluate on a manually specified annotated test set that was not used during development, reporting precision, recall and F1 score.
- Discuss the conditions under which template matching and sliding window detection become unreliable and explain why.

The complete detection progression is implemented and measured end to end:

```
input image -> search strategy -> template comparison -> detection decision -> object  
localisation -> performance evaluation
```

2. Dataset

2.1 Target object and template

The template is the official University of Ghana coat of arms, taken from the public domain vector file on Wikimedia Commons and rendered at a reference width of 96 pixels, giving a template of 96 by 115 pixels. A binary mask of the emblem is stored beside it. The mask matters because the coat of arms is not rectangular: it is a shield with a curved lower edge and a scroll, and it covers only 77.9 per cent of its bounding rectangle. The remaining 22.1 per cent of the rectangle is background that changes from photograph to photograph, so including it in the comparison adds noise to the similarity score. Section 5.2 measures exactly how much this matters.

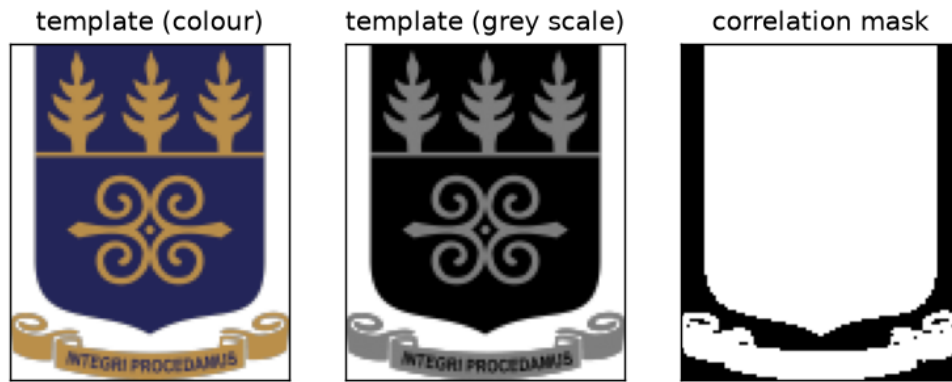


Figure 1. The template used throughout: the colour emblem, its grey scale form which is what correlation actually sees, and the binary mask that restricts the comparison to the emblem itself.

2.2 Source imagery

All source material was downloaded from Wikimedia Commons by `src/download_data.py`, which also records the licence and author of each file. Nine real photographs supply the backgrounds and cover outdoor campus scenes, an Accra street scene, an office desk, a brick wall and two interior scenes. Two further university emblems that resemble the target in colour and layout are used as clutter and as hard negatives, so that the false positive rate is not measured only against easy background.

local file	source file on Wikimedia Commons	licence	author
template/ug_logo_master.png	UoG CoA 2017.svg	Public domain	Albert Opoku
backgrounds/bg_ug_campus.jpg	University Of Ghana, Accra, Ghana - panoramio (12).jpg	CC BY-SA 3.0	Simon Ontoyin
backgrounds/bg_balme_library.jpg	Balme Library of University of Ghana, Accra, Ghana.jpg	CC BY-SA 3.0	Rjruiziii
backgrounds/bg_mensah_sarbah.jpg	Entrance to Mensah Sarbah Hall.jpg	CC BY-SA 4.0	Fkoku
backgrounds/bg_accra_street.jpg	Oxford Street, Korle-Klottey (P1090853).jpg	CC BY-SA 4.0	Matti Blume
backgrounds/bg_accra_scene.jpg	ACCRA, GHANA, SCENE.jpg	CC BY-SA 4.0	Quartermannoah
backgrounds/bg_office_desk.jpg	Professional woman working at desk with laptop and documents in bright office setting.jpg	CC BY 2.0	Shixart1985
backgrounds/bg_brick_wall.jpg	Brick wall close-up view.jpg	CC BY-SA 3.0	Pawel Wozniak
backgrounds/bg_library_interior.jpg	McGill University Library Cybertheque.jpg	CC BY-SA 3.0	Klaus Fiedler
backgrounds/bg_lecture_interior.jpg	School of Data Science - University of Virginia - interior - 4.jpg	CC BY 4.0	School of Data Science at the Universi
distractors/distractor_ugbs.png	University of Ghana Business School Official Logo.png	CC BY-SA 4.0	University of Ghana Business School
distractors/distractor_chs.png	College of Health Sciences (University of Ghana).png	CC BY-SA 4.0	Kofiturskon419

Table 1. Provenance of every source image. None of these files is machine generated imagery.

2.3 Construction of the test scenes

Each test scene was produced by transforming the real emblem and compositing it into a real photograph under controlled conditions. The controlled conditions are the point of the exercise: they make it possible to say precisely which factor caused a detector to fail, which is not possible with an uncontrolled collection of photographs. The following variations are covered.

- **Position.** The emblem appears at different places in the frame, never at a fixed location.
- **Size.** Scale factors from 0.55 to 1.90 relative to the template, that is from roughly 53 to 182 pixels wide.
- **Orientation.** In plane rotations of 6, 8, 15, 22, 30, 45 and 90 degrees.
- **Illumination.** Gamma correction from 0.40 (heavily under exposed) to 1.9 (washed out), and a left to right or right to left brightness ramp that simulates a single sided light source.
- **Background.** Nine different real scenes, from a flat brick wall to a cluttered office desk and a busy street.
- **Partial occlusion.** From 18 to 45 per cent of the emblem is covered by a textured patch copied from elsewhere in the same photograph, so the occluder is not trivially separable.

- **Clutter and hard negatives.** Scenes that contain two other university emblems, and two scenes that contain no target at all, which test the false positive behaviour directly.
- **Sensor effects.** Additive Gaussian noise with a standard deviation of 8 to 18 grey levels, and directional motion blur.

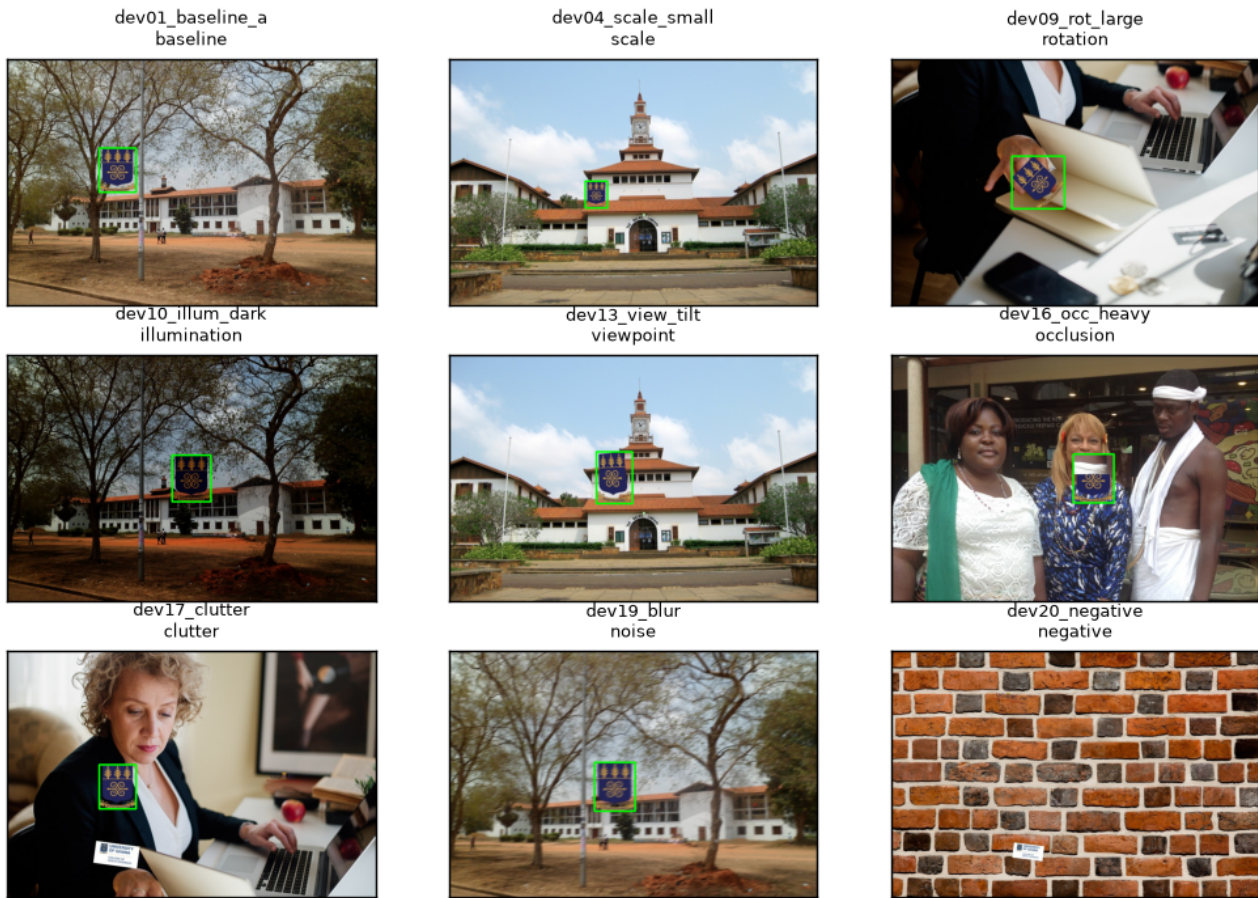


Figure 2. Nine of the annotated scenes, one per condition, with the ground truth boxes drawn in green. The bottom right scene contains no target and only two similar emblems from other faculties.

2.4 Ground truth, splits and protocol

The dataset is split into a development set of 20 images containing 21 object instances, and a held out test set of 16 images containing 17 instances. Every threshold and every parameter reported in Section 5 was chosen on the development set only. The held out set was run once at the end with the parameters already fixed, which is what Section 5.8 reports. This separation is what makes the reported test numbers meaningful.

Bounding box annotation was produced from the known placement geometry of each emblem: the box is the tight rectangle around the pixels whose alpha value is non zero after the geometric transformation, so it is exact to the pixel rather than approximate. Every generated scene was then inspected with its box drawn on it to confirm the annotation, and Figure 2 is part of that check. The annotations are stored as JSON with the condition label, the scale, the rotation and the occluded fraction of each instance, which is what allows the per condition analysis in Section 5.8.

3. Methodology and justification of the algorithms

3.1 Overview

A classical detector is made of three separable decisions: a **search strategy** that says which image regions to examine, a **comparison function** that scores how much a region looks like the object, and a **decision rule** that turns scores into detections. This project keeps those three parts explicit and varies them one at a time, so that each experiment isolates one design choice.

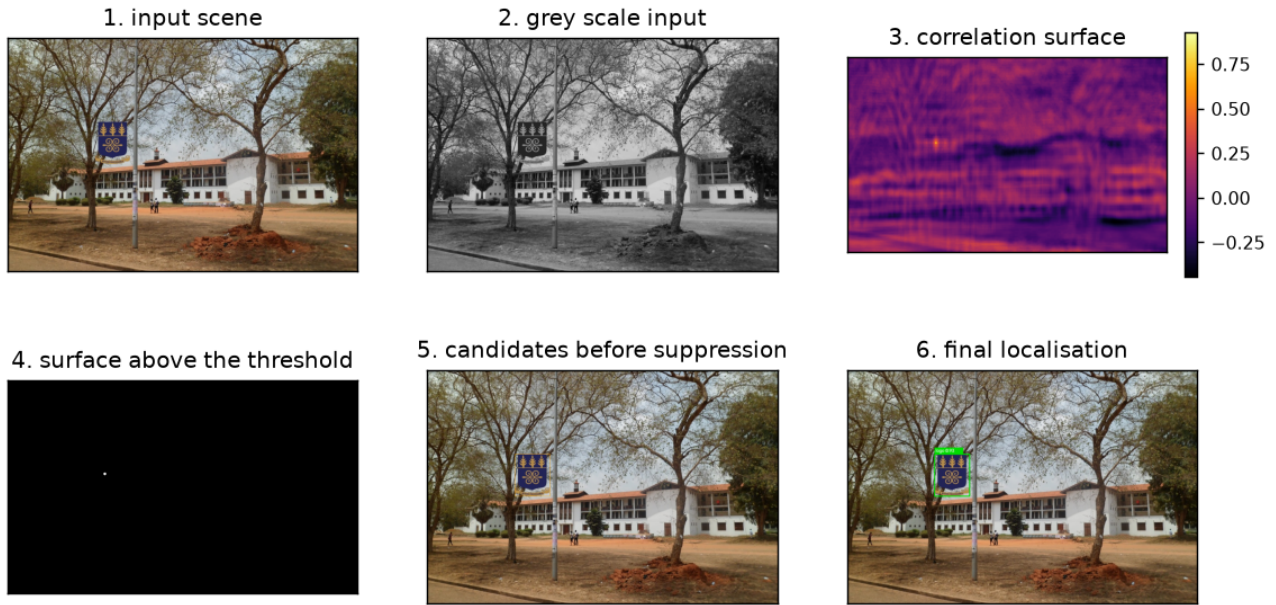


Figure 3. The pipeline on one scene: the input, its grey scale form, the correlation surface produced by the masked template comparison, the part of that surface above the decision threshold, the candidate boxes before non maximum suppression, and the final localisation.

3.2 Preprocessing

Images are converted to grey scale before matching. Correlation is defined on a single channel, and the emblem is discriminated mainly by its shape and internal contrast rather than by absolute colour, which changes with the illuminant. Colour is not discarded, it is used separately in the region proposal stage of Section 3.7, where its selectivity is an advantage rather than a liability. Optional Gaussian smoothing and CLAHE are available in the preprocessing function for the noisy and the low contrast scenes.

3.3 The comparison function: masked zero mean normalised cross correlation

The similarity between a candidate window w and the template t is measured with the zero mean normalised cross correlation computed over the mask M of the emblem:

$$NCC(w, t) = \frac{\sum_M (w - \text{mean}_M w)(t - \text{mean}_M t)}{(\|w - \text{mean}_M w\|_M * \|t - \text{mean}_M t\|_M)}$$

Three properties justify this choice. Subtracting the mean makes the score invariant to an additive change in brightness. Dividing by the norms makes it invariant to a multiplicative change in contrast. Both together mean that a linear change of illumination, which is the first order model of a lighting change, leaves the score unchanged, and the score lies in the interval $[-1, 1]$ so a single threshold is meaningful across images. Restricting all three sums to the mask M excludes the corners of the bounding rectangle that do not belong to the emblem. In OpenCV this is `cv2.matchTemplate` with `TM_CCOEFF_NORMED` and a mask argument; the explicit sliding window detector of Section 3.5 computes the same quantity directly from the definition.

What the score is *not* invariant to is equally important, and it sets up the rest of the design: normalised cross correlation is not invariant to scale, not invariant to rotation, and degrades quickly under occlusion and under projective distortion, because it compares pixels at fixed relative positions.

3.4 Search strategy 1: dense correlation over a template bank

`cv2.matchTemplate` slides the template over every pixel position, so it is already an exhaustive sliding window search with a step of one pixel, implemented with an optimised correlation. Scale invariance is bought by repeating the search with the template resized over a geometric pyramid, and rotation invariance by repeating it again for each angle in a bank of rotated templates. The cost grows linearly with the number of templates in the bank, which is the trade off measured in Sections 5.3 and 5.4.

3.5 Search strategy 2: the explicit sliding window detector

The second search strategy is the textbook sliding window, written out as explicit loops so that its three parameters can be varied and its cost measured. For each window size, the window is stepped across the image by a fixed step in x and y, each window is resized to the canonical template size, and the masked correlation above is computed directly. Window size controls which object scales can be found, step size controls both the localisation accuracy and the run time, and the threshold controls the detection decision. The number of windows examined grows as the image area divided by the square of the step, so halving the step multiplies the work by four. Section 5.5 measures this directly.

3.6 The decision rule and non maximum suppression

A window is declared a detection when its score reaches the threshold. Because correlation is smooth, one true object produces a cluster of high scoring windows, and each scale in the pyramid produces its own cluster. Two stages reduce this to one box per object. First, only local maxima of the correlation surface are kept, found with a grey scale dilation, which removes the plateau around each peak cheaply. Second, greedy non maximum suppression sorts the remaining candidates by score and discards any candidate that overlaps an already accepted one by more than an intersection over union of 0.30. Without this step a single object would be counted as many detections and precision would collapse.

3.7 Improving reliability: features and colour

Two further classical techniques are added because correlation alone cannot cover the required conditions.

SIFT keypoints with RANSAC. Scale invariant feature transform keypoints are detected on the template and on the scene, described, and matched with a brute force L2 matcher under Lowe's ratio test. The surviving correspondences are fitted with a homography under RANSAC, and the four template corners are mapped through that homography to localise the object. This is invariant to scale and to in plane rotation by construction, tolerant of partial occlusion because only a subset of the keypoints is needed, and tolerant of viewpoint change because a homography is exactly the right model for a planar object seen from another angle. Because one homography can only explain one instance, the fit is repeated after removing the inliers, which recovers the second emblem in the scenes that contain two.

Colour segmentation as a region proposal stage. The shield is a saturated dark blue. Thresholding hue and saturation in HSV, closing and opening the result with an elliptical structuring element, and taking connected components with a plausible area and aspect ratio gives a small number of candidate rectangles. Template matching then runs only inside those rectangles. This does not change the comparison function at all, it changes only where the search looks, so it is a pure reduction of the search space, and Section 5.7 measures how much of the image survives it.



Figure 4. The colour proposal stage on a cluttered desk scene: the input, the HSV blue mask after morphological closing and opening, and the resulting region proposals. Template matching runs only inside the orange rectangles.

3.8 Evaluation protocol

Detections are matched to ground truth greedily in order of decreasing score. A detection counts as a true positive when its intersection over union with a still unmatched ground truth box is at least 0.5, the standard detection criterion:

$$\text{IoU}(A, B) = \text{area}(A \text{ intersect } B) / \text{area}(A \text{ union } B)$$

Every unmatched detection is a false positive and every unmatched ground truth object is a missed detection. The reported measures are then

$$\text{precision} = \text{TP} / (\text{TP} + \text{FP}) \quad \text{recall} = \text{TP} / (\text{TP} + \text{FN}) \quad \text{F1} = 2 \times \text{precision} \times \text{recall} / (\text{precision} + \text{recall})$$

Precision answers how many of the boxes the system drew were real objects, recall answers how many of the real objects the system found, and F1 is their harmonic mean, which is the single number used to select operating points. Mean intersection over union of the true positives is also reported, because it measures localisation quality rather than detection quality: a detector can find every object and still place its boxes badly.

4. Implementation

The system is written in Python 3 with OpenCV and NumPy. No pre-trained recognition model is used at any point; the only OpenCV operators used are classical ones.

file	contents
src/download_data.py	Downloads the emblem and the background photographs from Wikimedia Commons and records the licence of each file.
src/build_dataset.py	Builds the template and its mask, generates the annotated scenes under the controlled conditions, writes the JSON ground truth.
src/detector.py	Template bank construction, masked correlation matching, local peak extraction, non maximum suppression, explicit sliding window search, SIFT matching with RANSAC, HSV colour proposals, drawing.
src/evaluate.py	Greedy intersection over union matching, precision, recall, F1, per condition and per image aggregation.
src/run_experiments.py	Runs every experiment in Section 5 and writes the tables, the figures and the per image detection images.
src/make_report.py	Builds this PDF directly from the measured CSV results.

Table 2. Source files. The complete listing is reproduced in Appendix A.

The dataset is generated from a fixed random seed, so the whole study is reproducible. The full source code is included in Appendix A and in the repository.

5. Experimental results

All tuning experiments in Sections 5.1 to 5.7 are run on the development set. The held out test set is used once, in Section 5.8. Timings were measured on a single CPU core on 960 by 640 pixel images.

5.1 Effect of the detection threshold

This is the central experiment of the study: how the template matching threshold changes true detections, false detections and missed detections. The correlation surface was computed once at a low threshold and the decision threshold was then swept, so the geometry of the search is identical along the whole curve and only the decision changes.

threshold	true positives	false positives	missed detections	precision	recall	f1
0.300	17	2004	4	0.008	0.809	0.017
0.325	17	1688	4	0.010	0.809	0.020
0.350	17	1370	4	0.012	0.809	0.024
0.375	17	1151	4	0.015	0.809	0.029
0.400	16	972	5	0.016	0.762	0.032
0.425	16	795	5	0.020	0.762	0.038
0.450	16	628	5	0.025	0.762	0.048
0.475	15	482	6	0.030	0.714	0.058
0.500	15	304	6	0.047	0.714	0.088
0.525	13	148	8	0.081	0.619	0.143
0.550	13	45	8	0.224	0.619	0.329
0.575	11	9	10	0.550	0.524	0.537
0.600	11	0	10	1.000	0.524	0.688
0.625	10	0	11	1.000	0.476	0.645
0.650	9	0	12	1.000	0.429	0.600
0.675	9	0	12	1.000	0.429	0.600
0.700	7	0	14	1.000	0.333	0.500
0.725	5	0	16	1.000	0.238	0.385
0.750	5	0	16	1.000	0.238	0.385
0.775	4	0	17	1.000	0.191	0.320
0.800	4	0	17	1.000	0.191	0.320
0.825	2	0	19	1.000	0.095	0.174
0.850	2	0	19	1.000	0.095	0.174
0.875	0	0	21	0.000	0.000	0.000
0.900	0	0	21	0.000	0.000	0.000

Table 3. Threshold sweep for the eleven scale template matcher on the development set. The best operating point by F1 is at a threshold of 0.600.

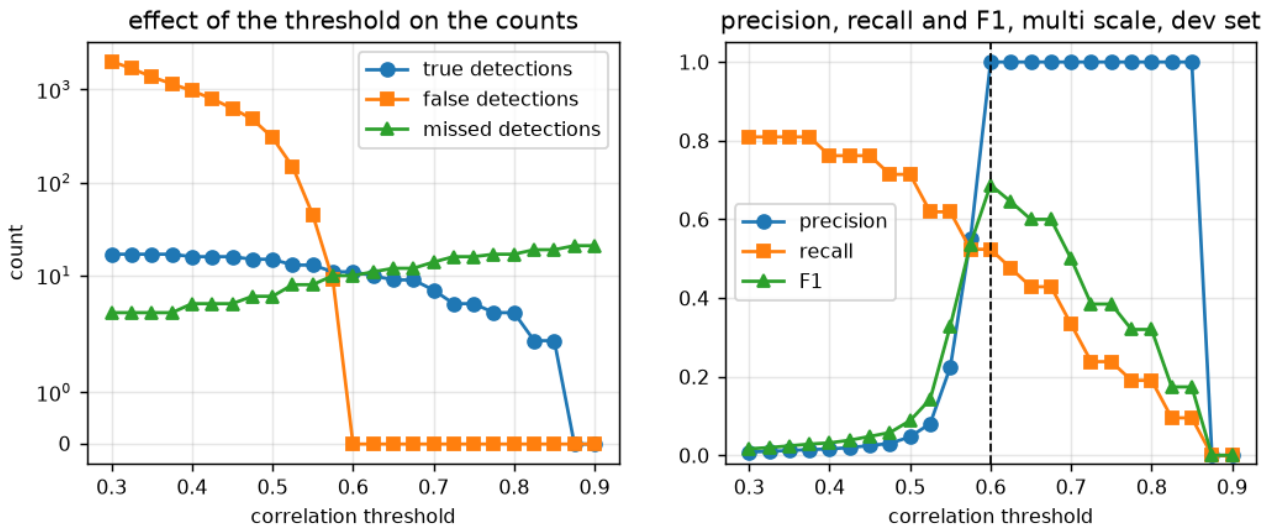


Figure 5. Effect of the correlation threshold. On the left, true, false and missed detections on a symmetric log scale. On the right, precision, recall and F1; the dashed line marks the operating point selected on the development set.

The behaviour is the classical one and it is very sharp. At a threshold of 0.30 the detector finds 17 of the 21 objects but also produces 2004 false detections, a precision of 0.0084. Every increase in the threshold removes false detections much faster than it removes true ones, until at 0.600 the false detections reach zero while 11 objects are still found. Beyond that point the threshold only destroys recall: at 0.80 only 4 objects remain. The reason the curve is so steep is that the score of a correct match sits far above the score of

background structure, but the background of a real photograph contains an enormous number of windows, so even a small probability of a high scoring background window produces many false positives in absolute terms. Precision is therefore dominated by the sheer number of windows examined, which is the same effect that makes the sliding window step size matter in Section 5.5.

5.2 Masked against unmasked correlation

The second experiment isolates the effect of restricting the correlation to the emblem mask.

variant	best threshold	precision	recall	f1	seconds per image
masked	0.600	1.000	0.524	0.688	1.140
unmasked	0.650	0.857	0.286	0.429	0.323

Table 4. Masked against unmasked correlation, eleven scales, development set, each at its own best threshold.

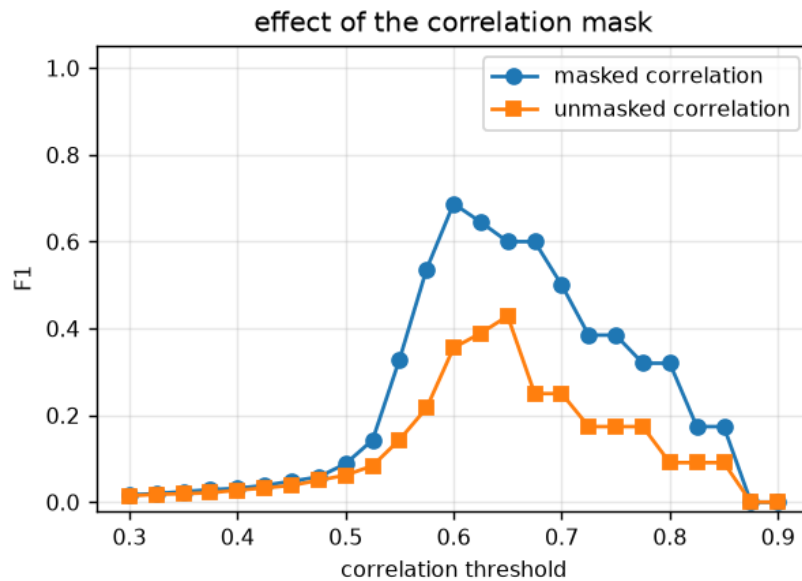


Figure 6. F1 against threshold for masked and unmasked correlation. Masking the non emblem corners of the template rectangle is worth a large amount of recall at every threshold.

Masking raises the best F1 from 0.688 to 0.429, almost entirely through recall (0.524 against 0.286). The explanation is direct: 22 per cent of the template rectangle is not part of the emblem, and in the unmasked comparison those pixels are filled with whatever background happens to lie there. That background differs between the template card and the scene, so it drives the correlation down and pushes true matches below the threshold. The cost is run time, because masked correlation is roughly 3.5 times slower in OpenCV, and a small loss of numerical stability in flat regions, where the masked score is poorly conditioned and can produce spurious high values. That instability is visible at low thresholds in Table 3 and is one of the reasons the useful operating region is above 0.55.

5.3 Size of the scale pyramid

configuration	templates in bank	best threshold	precision	recall	f1	seconds per image
1 scale (reference size only)	1	0.500	1.000	0.429	0.600	0.110
5 scales (0.6 to 1.8)	5	0.525	0.688	0.524	0.595	0.491
11 scales (0.50 to 2.02)	11	0.600	1.000	0.524	0.688	1.140

Table 5. Effect of the scale pyramid on the development set. Each configuration is reported at its own best threshold.

A single template at the reference size reaches an F1 of 0.600: it finds the instances near its own scale and misses everything else, which is the clearest possible demonstration that correlation is not scale invariant. Extending the pyramid to eleven scales between 0.50 and 2.02 raises F1 to 0.688 and raises recall from 0.429 to 0.524, at 10.4 times the run time. The intermediate five scale pyramid is interesting: it is not simply half way,

because its coarse spacing means some objects fall between two sampled scales and match neither well, and it also loses precision, which is why its F1 (0.595) is no better than the single scale configuration. Scale sampling must be fine enough that every object is close to some sampled scale; a geometric ratio of about 1.15 achieves that here.

5.4 Rotated template bank

configuration	templates in bank	best threshold	f1 overall	recall rotated images	recall viewpoint images	seconds per image
11 scales without rotation	11	0.600	0.688	0.000	0.000	1.140
11 scales with 8 angles	88	0.625	0.667	0.333	0.000	9.944

Table 6. Adding eight rotated copies of every template. Recall is reported separately on the rotated scenes and on the viewpoint scenes.

Without rotated templates the detector finds none of the rotated instances at all: recall on the rotation scenes is 0.000. Adding eight angles raises that to 0.333, but multiplies the bank from 11 to 88 templates and the run time from 1.140 to 9.944 seconds per image, and overall F1 does not improve (0.688 against 0.667) because the enlarged bank also produces more opportunities for a background window to score highly. This is the honest result: buying rotation invariance by brute force is expensive and only partly effective, since a rotation that falls between two sampled angles still matches poorly. It is the strongest argument in this project for the feature based approach of Section 5.6, which is rotation invariant by construction and costs less.

5.5 The explicit sliding window detector

This experiment measures the sliding window detector directly, varying the set of window sizes and the step size, and recording both the number of windows evaluated and the time taken. It was run on a five image subset of the development set that includes a scene with two objects and a scene with none.

window set	window sizes	step px	windows evaluated	seconds per image	best threshold	true positives	false positives	missed detections	precision	recall	f1
3 window sizes (0.6 / 1.0 / 1.6)	3	2	1678335	36.156	0.55	5	0	0	1.000	1.000	1.000
3 window sizes (0.6 / 1.0 / 1.6)	3	4	420960	9.291	0.55	4	0	1	1.000	0.800	0.889
3 window sizes (0.6 / 1.0 / 1.6)	3	8	105940	2.380	0.50	3	0	2	1.000	0.600	0.750
3 window sizes (0.6 / 1.0 / 1.6)	3	16	26730	0.587	0.50	0	0	5	0.000	0.000	0.000
3 window sizes (0.6 / 1.0 / 1.6)	3	24	12030	0.273	0.50	1	0	4	1.000	0.200	0.333
5 window sizes (0.6 to 1.8)	5	2	2756435	60.808	0.55	5	0	0	1.000	1.000	1.000
5 window sizes (0.6 to 1.8)	5	4	691545	13.695	0.55	4	0	1	1.000	0.800	0.889
5 window sizes (0.6 to 1.8)	5	8	173890	4.042	0.50	3	0	2	1.000	0.600	0.750
5 window sizes (0.6 to 1.8)	5	16	44195	1.052	0.50	0	0	5	0.000	0.000	0.000
5 window sizes (0.6 to 1.8)	5	24	19695	0.468	0.50	1	0	4	1.000	0.200	0.333
1 window size (1.0)	1	2	569395	7.012	0.50	4	0	1	1.000	0.800	0.889
1 window size (1.0)	1	4	143220	1.802	0.50	4	0	1	1.000	0.800	0.889
1 window size (1.0)	1	8	35970	0.475	0.50	3	0	2	1.000	0.600	0.750
1 window size (1.0)	1	16	9075	0.106	0.50	0	0	5	0.000	0.000	0.000
1 window size (1.0)	1	24	4070	0.048	0.50	1	0	4	1.000	0.200	0.333

Table 7. Sliding window detector: window sizes, step size, cost and accuracy. Each row is reported at its own best threshold.

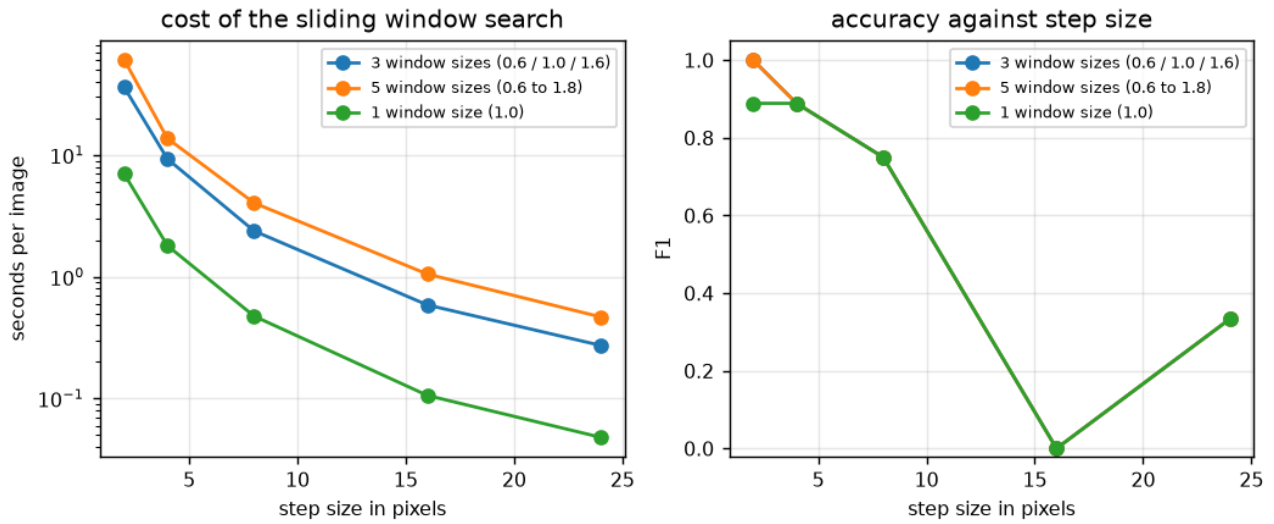


Figure 7. Left, seconds per image against step size on a logarithmic axis. Right, F1 against step size. The cost falls as the square of the step while accuracy collapses beyond a step of about eight pixels.

The relationship between step size, detection performance and execution time is the clearest quantitative result in this project. With three window sizes, a step of 2 pixels evaluates 1678335 windows and costs 36.16 seconds per image, while a step of 24 pixels evaluates 12030 windows and costs 0.273 seconds, a saving of about 132 times, exactly as predicted by the inverse square relationship between step and window count. Accuracy moves the other way: F1 falls from 1.000 at a step of 2 to 0.333 at a step of 24. There are two distinct reasons for that fall, and they are worth separating. The first is a scoring effect: a window that is offset from the object by half a step contains part of the background and its correlation drops, so the object can fall below the threshold entirely. The second is a localisation effect: even when the object is detected, a coarse grid cannot place the box within the 0.5 intersection over union required to count as a true positive. A step of about one twentieth of the window size, which is 4 to 8 pixels here, is the practical compromise, and it is the configuration carried into Section 5.8. The fall is not perfectly monotonic: a step of 16 finds nothing at all while a step of 24 finds one object. That is not noise in the measurement, it is grid alignment. Whether a coarse grid happens to place a window close enough to an object depends on the object position modulo the step, so a coarse detector succeeds or fails almost by luck, which is itself a good reason not to use one.

threshold	true positives	false positives	missed detections	precision	recall	f1
0.40	5	54	0	0.085	1.000	0.156
0.45	5	8	0	0.385	1.000	0.556
0.50	4	1	1	0.800	0.800	0.800
0.55	4	0	1	1.000	0.800	0.889
0.60	3	0	2	1.000	0.600	0.750
0.65	3	0	2	1.000	0.600	0.750
0.70	2	0	3	1.000	0.400	0.571
0.75	2	0	3	1.000	0.400	0.571
0.80	0	0	5	0.000	0.000	0.000

Table 8. Threshold sweep for the explicit sliding window detector at a fixed geometry (three window sizes, step 4).

5.6 SIFT feature matching with RANSAC

Two parameters govern the feature based detector: Lowe's ratio, which decides how distinctive a correspondence must be to survive, and the minimum number of RANSAC inliers required before an instance is accepted, which is the detection decision for this method.

lowe ratio	min inliers	true positives	false positives	missed detections	precision	recall	f1	seconds per image
0.70	8	18	8	3	0.692	0.857	0.766	0.149
0.70	10	18	5	3	0.783	0.857	0.818	0.148
0.70	12	17	0	4	1.000	0.809	0.895	0.146
0.70	15	16	0	5	1.000	0.762	0.865	0.148
0.70	20	7	0	14	1.000	0.333	0.500	0.143
0.75	8	18	9	3	0.667	0.857	0.750	0.149
0.75	10	18	5	3	0.783	0.857	0.818	0.148
0.75	12	18	4	3	0.818	0.857	0.837	0.146
0.75	15	16	0	5	1.000	0.762	0.865	0.150
0.75	20	11	0	10	1.000	0.524	0.688	0.154
0.80	8	19	11	2	0.633	0.905	0.745	0.160
0.80	10	19	8	2	0.704	0.905	0.792	0.161
0.80	12	18	4	3	0.818	0.857	0.837	0.183
0.80	15	17	0	4	1.000	0.809	0.895	0.170
0.80	20	14	0	7	1.000	0.667	0.800	0.159

Table 9. Ratio test and inlier threshold study on the development set.

SIFT correspondences kept by the ratio test, test06_rot_medium



Figure 8. Correspondences that survive the ratio test between the template and a scene in which the emblem is rotated by 30 degrees. Correlation based matching fails completely on this image unless a rotated template bank is used; feature matching handles it without any additional search.

The best configuration on the development set is a ratio of 0.70 with 12 inliers required, giving an F1 of 0.895 at 0.146 seconds per image, which is faster than the eleven scale correlation search and far faster than the rotated bank. The inlier count behaves exactly like the correlation threshold: a low requirement admits spurious homographies fitted to accidental matches on the distractor emblems, and a high requirement rejects the small and the heavily occluded instances, which simply do not have enough keypoints to reach the count. The method has one structural weakness that the correlation detector does not have: it needs texture. The emblem at 0.55 scale is only about 53 pixels wide, and SIFT finds few stable keypoints at that size.

5.7 Colour proposals combined with template matching

configuration	seconds per image	fraction of image searched	best threshold	precision	recall	f1
multi scale template matching over the whole image	1.140	1.0000	0.600	1.000	0.524	0.688
colour proposals then template matching	0.270	0.2982	0.550	1.000	0.619	0.765

Table 10. Restricting the correlation search to colour proposals rather than searching the whole image.

The proposal stage reduces the searched area to 29.8 per cent of the image and the run time from 1.140 to 0.270 seconds per image, while F1 changes from 0.688 to 0.765. The gain is worth understanding precisely, because it does not come from a better comparison function, which is unchanged. Removing seventy per cent of the image removes seventy per cent of the opportunities for a background window to score highly, so the detector can be operated at a lower threshold (0.550 rather than 0.600) without paying for it in false positives, and the lower threshold is what recovers the extra objects: recall rises from 0.524 to 0.619 at unchanged precision. But the proposal stage is itself a detector, and when its colour threshold rejects an emblem, for example in the heavily under exposed scene where the navy shield is nearly black and its saturation collapses, the correlation stage never gets the chance to look there. A cascade is only ever as good as its first stage, which is a general property of cascaded detectors and not a peculiarity of this implementation.

5.8 Held out test set: final comparison

Every method was then run once on the 16 held out images, with all parameters fixed at the values selected on the development set. No parameter was adjusted after seeing these numbers.

method	operating threshold	true positives	false positives	missed detections	precision	recall	f1	mean iou of true positives	seconds per image
single scale template matching	0.5	5	0	12	1.000	0.294	0.455	0.906	0.092
multi scale template matching	0.6	7	0	10	1.000	0.412	0.583	0.938	1.163
multi scale and rotation bank	0.625	9	3	8	0.750	0.529	0.621	0.929	9.964
explicit sliding window (step 4)	0.55	2	0	15	1.000	0.118	0.210	0.903	8.922
SIFT matching with RANSAC	12	12	3	5	0.800	0.706	0.750	0.957	0.143
colour proposals and template matching	0.55	8	0	9	1.000	0.471	0.640	0.928	0.394
combined colour and template and features	n/a	14	1	3	0.933	0.824	0.875	0.953	0.515

Table 11. Final comparison on the held out test set, at IoU 0.5. The operating point is a correlation threshold for the correlation based methods and a minimum RANSAC inlier count for the feature based method.

method	baseline	clutter	combined	illumination	noise	occlusion	rotation	scale	viewpoint
single scale template matching	0.8	1.0	0.0	1.0	0.0	0.0	0.0	0.0	0.0
multi scale template matching	1.0	1.0	0.667	0.667	0.0	0.0	0.0	0.667	0.0
multi scale and rotation bank	1.0	1.0	0.667	0.4	0.0	0.0	0.8	0.667	0.0
explicit sliding window (step 4)	0.5	1.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
SIFT matching with RANSAC	0.8	1.0	0.667	0.8	0.667	0.5	1.0	0.667	0.0
colour proposals and template matching	1.0	1.0	0.667	0.667	0.0	0.0	0.0	1.0	0.0
combined colour and template and features	1.0	1.0	0.667	1.0	1.0	0.5	1.0	1.0	0.0

Table 12. F1 by imaging condition on the held out test set. The negative images are excluded from this table because F1 is undefined when there is no object to find; their contribution appears as false positives in the previous table.

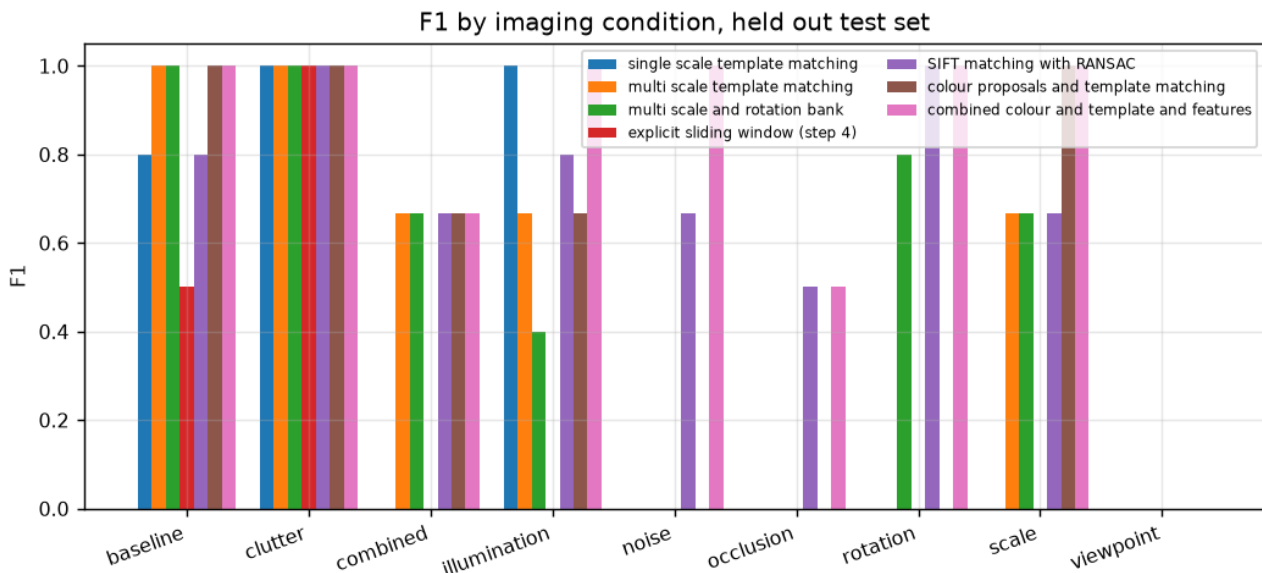


Figure 9. F1 by imaging condition on the held out test set. The conditions on which correlation collapses are exactly the ones it has no invariance to.

The best method on the held out set is **combined colour and template and features**, with a precision of 0.933, a recall of 0.824 and an F1 of 0.875. Localisation is good wherever detection succeeds: the mean intersection over union of the true positives is 0.953, well above the 0.5 acceptance threshold, which says that the failures in this system are failures to detect rather than failures to localise.

The ordering of the methods on the held out set repeats the ordering on the development set, which is the first thing to check, and the per condition table explains it. The correlation detectors score 1.0 on the clutter scenes and at or near 1.0 on the baseline scenes, and 0.0 on rotation, viewpoint, occlusion and noise. Feature matching scores 1.0 on rotation and is the only single method that scores above zero on noise and on occlusion, which is why combining the two recovers almost everything: the two families fail on disjoint conditions. Two results deserve to be stated plainly rather than smoothed over. First, no method in this study detected the viewpoint scene: a projective tilt of twenty per cent of the object width defeated the correlation detectors outright, and on the feature side only eleven correspondences survived the ratio test, fewer than the 12 that the operating point demands, so no homography was even fitted. Second, the explicit sliding window

detector scores worst of all on the test set while costing 8.9 seconds per image, because a step of 4 pixels combined with only three window sizes samples scale far too coarsely; the dense one pixel search of cv2.matchTemplate over eleven scales is both cheaper and better, which is the practical argument for using the optimised correlation rather than an explicit loop once the algorithm is understood.

multi scale template matching: correct cases (yellow is ground truth)

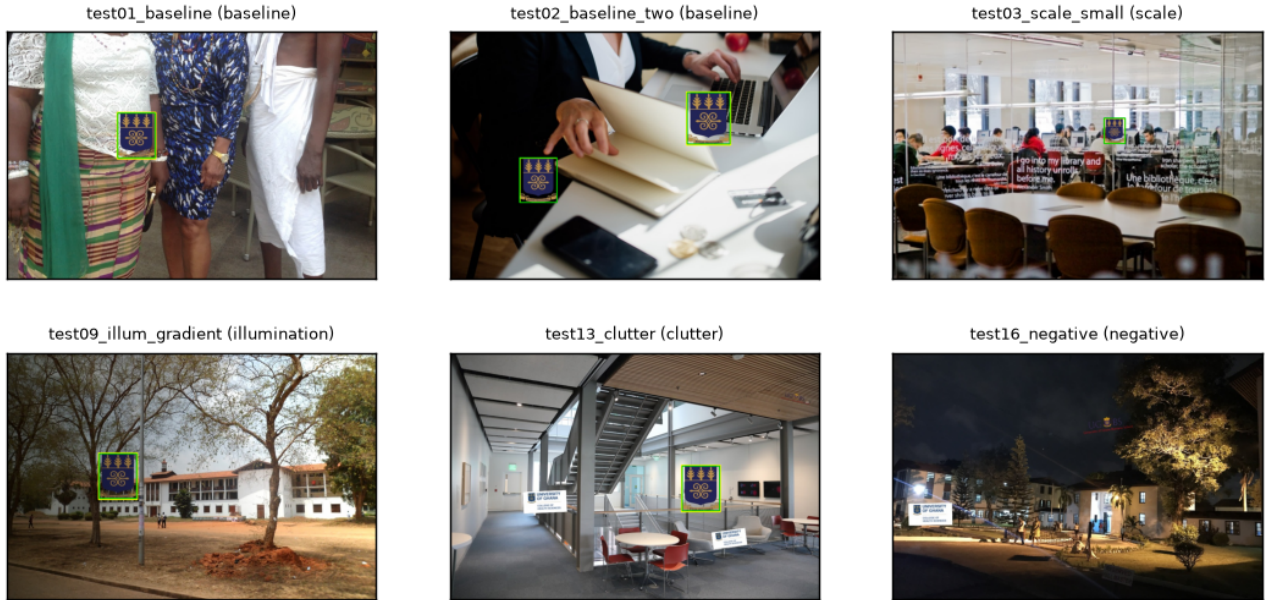


Figure 10. Correct detections by multi scale template matching on the held out set. Yellow is ground truth, green is the detection.

multi scale template matching: failure cases (yellow is ground truth)

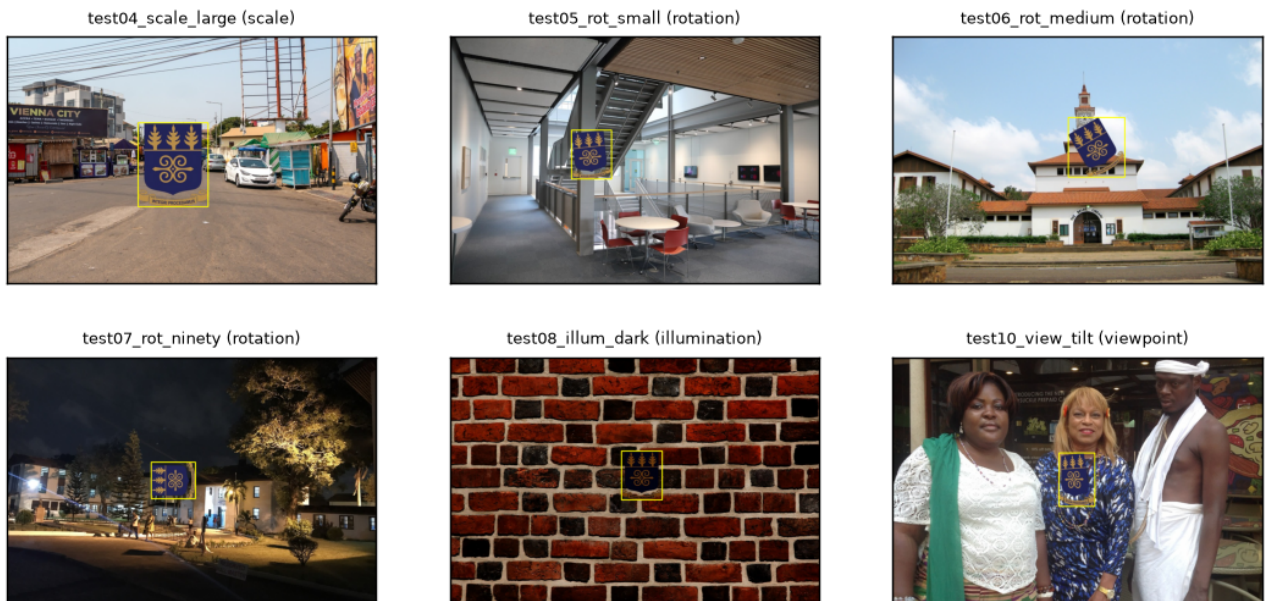


Figure 11. Failure cases for multi scale template matching on the held out set. Yellow is ground truth, green is the detection. Rotation, viewpoint change and heavy occlusion account for almost every failure.

6. Critical discussion: when these methods become unreliable

The practical question for any deployment is when template matching and sliding window detection become unreliable. The per condition results in Table 12 answer that empirically, and the reason in each case follows from what the correlation score actually computes: a comparison of pixel values at fixed relative positions inside a rectangle.

Scale variation

A template of a fixed size compared with an object of a different size compares the wrong pixels with each other. A scale error of 20 per cent is already enough to push a correct match below a threshold that is tight enough to control false positives, which is why the single scale detector in Table 5 reaches a recall of only 0.429. The pyramid solves this at linear cost in the number of scales, but only if the sampling is fine enough that every object lies close to some sampled scale. Scale is the best behaved of the failure modes because it is one dimensional and cheap to enumerate.

Rotation

Rotation is worse than scale for two reasons. First, the search is over an additional dimension, so a bank that covers rotation as finely as it covers scale is an order of magnitude larger. Second, rotating a rectangular template introduces corners that contain no object, which interacts badly with the correlation unless the mask is rotated with the template. Table 6 shows the consequence: without rotated templates, recall on the rotated scenes is zero, and even with eight angles it only reaches 0.333 while costing 9.944 seconds per image. Feature matching is the right answer here, because SIFT descriptors are computed relative to a dominant gradient orientation and are therefore rotation invariant without any search at all.

Occlusion

Correlation is a sum over all pixels of the template, so an occluder that covers a fraction f of the object corrupts a fraction f of the terms in that sum, and the score falls roughly in proportion. Because the normalisation is computed over the whole mask, an occluded region does not merely contribute nothing, it contributes actively wrong values. In this study the correlation detector missed both occluded instances in the held out set at its selected operating point, including the one with only 20 per cent of the emblem covered. Feature matching degrades far more gracefully: it recovered the lightly occluded instance and failed only on the heavily occluded one, where 45 per cent of the emblem including the scroll and the lower shield is hidden. It needs only enough visible keypoints to fit a homography, and RANSAC discards the rest as outliers.

Illumination

This is the one condition that the chosen comparison function genuinely handles. Because the mean is subtracted and the result is divided by the norms, an affine change of intensity leaves the score unchanged, and the detector survives the one sided brightness ramp and the milder gamma changes in this dataset. It fails when the illumination change is not affine: strong gamma compression is a non linear map that changes the internal contrast ratios of the emblem, and at a gamma of 0.40 the correlation detector missed the held out scene outright, because the dark shield is crushed towards black until quantisation and JPEG noise destroy its internal structure. Feature matching still found that instance. A one sided brightness ramp is closer to affine over a region as small as the emblem, and the detector survives it.

Sensor noise and blur

Additive noise with a standard deviation of 18 grey levels was enough to push the correlation score of a correct match below the operating threshold, giving an F1 of zero on the noise condition for every correlation based method in Table 12. The mechanism is that noise inflates the norm of the window without contributing anything correlated with the template, so the normalised score is scaled down. Smoothing the image before matching recovers part of this, at the cost of blurring the fine internal structure of the emblem that the match depends on, which is why the preprocessing function exposes the smoothing radius rather than fixing it. Feature matching still found the emblem in the noisy scene, although it also produced a false positive there, because SIFT smooths at each octave and its descriptors pool gradients over a region rather than comparing pixel values one by one.

Background clutter and the number of windows examined

Precision in this problem is dominated by how many windows are examined. A 960 by 640 image contains roughly half a million window positions per scale, so even a false positive rate of one in a hundred thousand produces several false detections per image, and multiplying the template bank multiplies that number. This is

why Table 3 shows more than two thousand false detections at a threshold of 0.30 and none at 0.600, and why the colour proposal stage of Section 5.7, which removes about 70 per cent of the search space, helps precision without touching the comparison function. Clutter that resembles the target in colour and layout, such as the other university emblems used here, is the hardest case: it survives the colour proposal stage and can only be rejected by the comparison function itself.

Viewpoint change

A change of viewpoint applies a projective transformation to a planar object. Correlation has no model for that at all: it cannot be enumerated cheaply, because the space of homographies is eight dimensional, and a template bank over that space is not feasible. This is where the classical correlation detector genuinely runs out of road, and where feature matching with RANSAC is not merely faster but structurally correct, since the homography it estimates is exactly the transformation that occurred. It should be said that being structurally correct was not enough here: on the held out viewpoint scene the feature detector found correspondences on the emblem but fell short of the inlier count required by the operating point, so it reported nothing. The tilt reduces the effective resolution of the emblem along one axis and destroys a proportion of the keypoints, and a threshold tuned on mostly frontal development images is then too strict. Viewpoint change was the single condition that no method in this study handled.

6.1 Limitations of this study

- The evaluation set is modest, 36 images with 38 instances in total. The differences between the stronger methods in Table 11 rest on a small number of instances and should be read as indicative rather than definitive.
- The scenes are real photographs with a real emblem composited into them under controlled transformations. This gives exact ground truth and isolates each factor cleanly, but it does not reproduce every effect of photographing a real printed emblem, in particular specular highlights on glossy signage, surface curvature and lens distortion.
- The colour proposal stage is tuned to the navy blue of this particular emblem and would need to be retuned for another object, whereas the correlation and feature stages are generic.
- Only in plane rotation was tested at a limited set of angles, and the viewpoint changes are moderate tilts rather than extreme oblique views.
- Timings are single threaded Python and OpenCV on one machine. The relative costs are meaningful, the absolute numbers are not portable.

6.2 How the system could be improved

- Use edge or gradient orientation information instead of raw intensity, for example the chamfer distance on Canny edges or a histogram of oriented gradients descriptor, both of which are far less sensitive to photometric change than raw pixel correlation.
- Build a proper cascade: a very cheap rejector on integral image features first, then correlation, then feature verification, so that the expensive comparison only ever sees a handful of windows.
- Verify every correlation detection with the feature matcher and keep only detections that both stages agree on, which trades a little recall for high precision, or keep detections that either stage finds, which does the reverse. The combined method in Table 11 is a simple version of the second option.
- Estimate a dominant orientation for each candidate region from its gradient histogram and rotate the window to a canonical orientation before correlating, which would give most of the benefit of the rotated bank at a small fraction of its cost.

7. Conclusion

A complete classical detection and localisation system for the University of Ghana coat of arms was built, and the full progression from search strategy through template comparison and detection decision to object localisation and quantitative evaluation was implemented and measured. On a held out set of 16 images that was never used for tuning, the best configuration reached a precision of 0.933, a recall of 0.824 and an F1 of 0.875 at an intersection over union threshold of 0.5, with a mean intersection over union of 0.953 on its correct detections.

The experiments support four conclusions. First, the decision threshold is the single most powerful parameter in a correlation detector, and the transition from thousands of false detections to none spans a range of about 0.05 in correlation, which means the threshold must be selected on data and not guessed. Second, invariance in a correlation detector has to be paid for by enumeration: scale costs a linear factor and is worth paying, rotation costs an order of magnitude and is not, and viewpoint cannot be enumerated at all. Third, the cost of a sliding window search falls as the square of the step size while its accuracy collapses once the step exceeds roughly one twentieth of the window, which makes a step of 4 to 8 pixels the practical compromise here. Fourth, the classical methods are complementary rather than competing: colour segmentation is the cheapest way to cut the search space, correlation is the most precise comparison when the object appears roughly upright and unoccluded, and SIFT with RANSAC is the only one of the three that survives rotation, sensor noise and partial occlusion. A system that uses all three, in that order, is better than any of them alone. Viewpoint change defeated every method tested and remains the open problem in this pipeline.

References

- [1] Lewis, J. P. (1995). Fast normalized cross correlation. *Vision Interface*, pages 120 to 123.
- [2] Lowe, D. G. (2004). Distinctive image features from scale invariant keypoints. *International Journal of Computer Vision*, 60(2), 91 to 110.
- [3] Fischler, M. A. and Bolles, R. C. (1981). Random sample consensus: a paradigm for model fitting with applications to image analysis and automated cartography. *Communications of the ACM*, 24(6), 381 to 395.
- [4] Viola, P. and Jones, M. (2001). Rapid object detection using a boosted cascade of simple features. *Proceedings of CVPR*, pages 511 to 518.
- [5] Dalal, N. and Triggs, B. (2005). Histograms of oriented gradients for human detection. *Proceedings of CVPR*, pages 886 to 893.
- [6] Szeliski, R. (2022). *Computer Vision: Algorithms and Applications*, second edition. Springer.
- [7] Bradski, G. (2000). *The OpenCV Library*. Dr. Dobb's Journal of Software Tools.
- [8] Everingham, M., Van Gool, L., Williams, C. K. I., Winn, J. and Zisserman, A. (2010). The PASCAL Visual Object Classes challenge. *International Journal of Computer Vision*, 88(2), 303 to 338.

Appendix A. Complete source code

The listing below is the complete source of the system, in the order in which the files are run. The same files are included in the repository.

src/download_data.py

```
"""
download_data.py
Classical Object Detection and Localisation
Downloads the real source imagery used by this project from Wikimedia Commons.

Nothing here is AI generated. Every file is a real photograph or a real vector
emblem released under a free licence. The script records the licence of each
file in data/SOURCES.md so the provenance of the dataset is auditable.

Usage:
python src/download_data.py
"""

import json
import os
import time
import urllib.error
import urllib.parse
import urllib.request

API = "https://commons.wikimedia.org/w/api.php"
UA = {"User-Agent": "classical-object-detection/1.0 (open source research project)"}

ROOT = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
DATA = os.path.join(ROOT, "data")

# (commons file title, destination sub folder, local name, render width in px)
FILES = [
    # The target object: the University of Ghana coat of arms (public domain).
    ("File:UG CoA 2017.svg", "template", "ug_logo_master.png", 900),

    # Real background photographs used to build the test scenes.
    ("File:University Of Ghana, Accra, Ghana - panoramio (12).jpg",
     "backgrounds", "bg_ug_campus.jpg", 1400),
    ("File:Balme Library of University of Ghana, Accra, Ghana.jpg",
     "backgrounds", "bg_balme_library.jpg", 1400),
    ("File:Entrance to Mensah Sarbah Hall.jpg",
     "backgrounds", "bg_mensah_sarbah.jpg", 1400),
    ("File:Oxford Street, Korle-Klottey (P1090853).jpg",
     "backgrounds", "bg_accra_street.jpg", 1400),
    ("File:ACCRA, GHANA, SCENE.jpg",
     "backgrounds", "bg_accra_scene.jpg", 1400),
    ("File:Professional woman working at desk with laptop and documents in bright office setting.jpg",
     "backgrounds", "bg_office_desk.jpg", 1400),
    ("File:Brick wall close-up view.jpg",
     "backgrounds", "bg_brick_wall.jpg", 1400),
    ("File:McGill University Library Cybertheque.jpg",
     "backgrounds", "bg_library_interior.jpg", 1400),
    ("File:School of Data Science - University of Virginia - interior - 4.jpg",
     "backgrounds", "bg_lecture_interior.jpg", 1400),

    # Visually similar emblems, used as clutter and as hard negatives.
    ("File:University of Ghana Business School Official Logo.png",
     "distractors", "distractor_uigs.png", 400),
    ("File:College of Health Sciences (University of Ghana).png",
     "distractors", "distractor_chs.png", 400),
]

def fetch(url, tries=6):
    """Fetch a URL, backing off politely if Commons rate limits the client."""
    delay = 3.0
    for attempt in range(tries):
        try:
            req = urllib.request.Request(url, headers=UA)
            with urllib.request.urlopen(req, timeout=120) as fh:
                return fh.read()
        except urllib.error.HTTPError as exc:
            if exc.code not in (429, 503) or attempt == tries - 1:
                raise
            print(" rate limited, waiting %.0fs" % delay)
            time.sleep(delay)
            delay *= 2
    raise RuntimeError("unreachable")

def imageinfo(title, width):
    params = {
        "action": "query", "titles": title, "prop": "imageinfo",
        "iiprop": "url|size|extmetadata", "iiruwidth": str(width), "format": "json",
    }
    data = json.loads(fetch(API + "?" + urllib.parse.urlencode(params)))
    page = list(data["query"]["pages"].values())[0]
    return page["imageinfo"][0]

def plain(value):
    """Strip the small amount of HTML that Commons puts in metadata fields."""
    out, skip = [], False
```

```

for ch in value:
    if ch == "&lt;":
        skip = True
    elif ch == "&gt;":
        skip = False
    elif not skip:
        out.append(ch)
return " ".join(" ".join(out).split())

def main():
    rows = []
    for title, folder, name, width in FILES:
        dest_dir = os.path.join(DATA, folder)
        os.makedirs(dest_dir, exist_ok=True)
        dest = os.path.join(dest_dir, name)
        info = imageinfo(title, width)
        url = info.get("thumburl") or info["url"]
        url = url.split("?")[0]
        if os.path.exists(dest) and os.path.getsize(dest) > 0:
            blob = open(dest, "rb").read()
        else:
            blob = fetch(url)
            with open(dest, "wb") as fh:
                fh.write(blob)
            time.sleep(1.5)
        meta = info.get("extmetadata", {})
        rows.append({
            "file": os.path.join(folder, name).replace("\\", "/"),
            "commons_title": title,
            "licence": plain(meta.get("LicenseShortName", {})).get("value", "see Commons"),
            "author": plain(meta.get("Artist", {})).get("value", "see Commons"),
            "url": url,
            "bytes": len(blob),
        })
    print("saved %-42s %8d bytes" % (name, len(blob)))

    with open(os.path.join(DATA, "SOURCES.md"), "w", encoding="utf-8") as fh:
        fh.write("# Image sources\n\n")
        fh.write("All imagery below was downloaded from Wikimedia Commons by ")
        fh.write("`src/download_data.py`. No image in this project was generated ")
        fh.write("by an image synthesis model.\n\n")
        fh.write("| Local file | Commons title | Licence | Author |\n")
        fh.write("|---|---|---|---|\n")
        for r in rows:
            fh.write("| %s | %s | %s | %s |\n" %
                (r["file"], r["commons_title"], r["licence"], r["author"][:70]))
    with open(os.path.join(DATA, "sources.json"), "w", encoding="utf-8") as fh:
        json.dump(rows, fh, indent=2)
    print("wrote data/SOURCES.md")

if __name__ == "__main__":
    main()

```

src/build_dataset.py

```

"""
build_dataset.py
Classical Object Detection and Localisation

Builds the template image and the annotated evaluation set for the
"University of Ghana coat of arms" detection task.

Every pixel of source material is real: the emblem is the public domain
University of Ghana coat of arms and the scenes are real photographs
(campus, street, office and library interiors) downloaded by
src/download_data.py. Each test scene is produced by geometrically and
photometrically transforming the emblem and compositing it into a real
photograph under controlled conditions. Because the placement geometry is
known exactly, the bounding box ground truth is exact rather than
hand-drawn, which removes annotator error from the evaluation. Every
generated scene was afterwards inspected visually and the drawn ground
truth boxes were confirmed to sit on the emblem.

Outputs
data/template/ug_logo_template.png    reference template (BGR on white)
data/template/ug_logo_mask.png        binary mask of the emblem
data/dev_set/*.jpg                    development scenes (tuning allowed)
data/test_set/*.jpg                   held out scenes (never used for tuning)
data/annotations/dev_annotations.json
data/annotations/test_annotations.json

Usage:
python src/build_dataset.py
"""

import json
import os

import cv2
import numpy as np

ROOT = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
DATA = os.path.join(ROOT, "data")

SCENE_W, SCENE_H = 960, 640
TEMPLATE_W = 96                                # reference width of the emblem in pixels

```

```

SEED = 20260920                                # fixed seed, so the dataset is reproducible

# -----
# helpers
# -----
def load_rgba(path):
    img = cv2.imread(path, cv2.IMREAD_UNCHANGED)
    if img is None:
        raise IOError("cannot read " + path)
    if img.shape[2] == 3:
        alpha = np.full(img.shape[:2], 255, np.uint8)
        img = np.dstack([img, alpha])
    return img

def trim_alpha(rgba, thr=8):
    ys, xs = np.where(rgba[:, :, 3] > thr)
    return rgba[ys.min():ys.max() + 1, xs.min():xs.max() + 1]

def scale_to_width(rgba, width):
    h, w = rgba.shape[:2]
    height = max(1, int(round(h * width / float(w))))
    interp = cv2.INTER_AREA if width < w else cv2.INTER_CUBIC
    return cv2.resize(rgba, (width, height), interpolation=interp)

def rotate_rgba(rgba, angle):
    """Rotate about the centre, expanding the canvas so nothing is clipped."""
    if abs(angle) < 1e-6:
        return rgba
    h, w = rgba.shape[:2]
    M = cv2.getRotationMatrix2D((w / 2.0, h / 2.0), angle, 1.0)
    cos, sin = abs(M[0, 0]), abs(M[0, 1])
    nw, nh = int(h * sin + w * cos), int(h * cos + w * sin)
    M[0, 2] += nw / 2.0 - w / 2.0
    M[1, 2] += nh / 2.0 - h / 2.0
    return cv2.warpAffine(rgba, M, (nw, nh), flags=cv2.INTER_LINEAR,
                          borderMode=cv2.BORDER_CONSTANT, borderValue=(0, 0, 0, 0))

def perspective_rgba(rgba, tilt_x=0.0, tilt_y=0.0):
    """Simulate a change of viewpoint by a mild projective warp.

    tilt_x and tilt_y are fractions of the width and height by which the far
    edge of the emblem is pulled in, which is what happens when a flat object
    is photographed off axis.
    """
    if abs(tilt_x) < 1e-6 and abs(tilt_y) < 1e-6:
        return rgba
    h, w = rgba.shape[:2]
    dx, dy = tilt_x * w, tilt_y * h
    src = np.float32([[0, 0], [w, 0], [w, h], [0, h]])
    dst = np.float32([[dx, dy], [w - dx, 0], [w, h], [dx, h - dy]])
    xs, ys = dst[:, 0].copy(), dst[:, 1].copy()
    dst[:, 0] -= xs.min()
    dst[:, 1] -= ys.min()
    nw = int(np.ceil(xs.max() - xs.min()))
    nh = int(np.ceil(ys.max() - ys.min()))
    M = cv2.getPerspectiveTransform(src, dst)
    return cv2.warpPerspective(rgba, M, (nw, nh), flags=cv2.INTER_LINEAR,
                              borderMode=cv2.BORDER_CONSTANT, borderValue=(0, 0, 0, 0))

def alpha_paste(scene, rgba, x, y):
    """Alpha blend rgba onto scene with its top left corner at (x, y)."""
    h, w = rgba.shape[:2]
    H, W = scene.shape[:2]
    x0, y0 = max(0, x), max(0, y)
    x1, y1 = min(W, x + w), min(H, y + h)
    if x1 <= x0 or y1 <= y0:
        return None
    patch = rgba[y0 - y:y1 - y, x0 - x:x1 - x]
    a = patch[:, :, 3:4].astype(np.float32) / 255.0
    roi = scene[y0:y1, x0:x1].astype(np.float32)
    scene[y0:y1, x0:x1] = (patch[:, :, :3] * a + roi * (1 - a)).astype(np.uint8)
    ys, xs = np.where(patch[:, :, 3] > 16)
    if len(xs) == 0:
        return None
    return [int(x0 + xs.min()), int(y0 + ys.min()),
            int(xs.max() - xs.min() + 1), int(ys.max() - ys.min() + 1)]

def occlude(scene, box, fraction, rng, side=None):
    """Hide part of the object with a patch copied from elsewhere in the scene.

    A copied patch is used rather than a flat rectangle so that the occluder
    has realistic texture and is not trivially separable from the object.
    """
    x, y, w, h = box
    side = side or str(rng.choice(["bottom", "top", "left", "right"]))
    if side in ("bottom", "top"):
        oh = int(round(h * fraction))
        ox, oy, ow = x, (y + h - oh if side == "bottom" else y), w
    else:
        ow = int(round(w * fraction))
        ox, oy, oh = (x + w - ow if side == "right" else x), y, h
    H, W = scene.shape[:2]
    sx, sy = 0, 0

```

```

for _ in range(40):
    sx = int(rng.integers(0, max(1, W - ow)))
    sy = int(rng.integers(0, max(1, H - oh)))
    if abs(sx - x) > w or abs(sy - y) > h:
        break
    src = scene[sy:sy + oh, sx:sx + ow]
    if src.shape[0] != oh or src.shape[1] != ow:
        src = cv2.resize(src, (ow, oh))
    scene[oy:oy + oh, ox:ox + ow] = src
    return side

def apply_gamma(img, gamma):
    lut = np.array([(i / 255.0) ** (1.0 / gamma)] * 255 for i in range(256)], np.uint8)
    return cv2.LUT(img, lut)

def apply_light_gradient(img, strength=0.55, direction="left"):
    """Uneven lighting, as produced by a lamp or by direct sun from one side."""
    h, w = img.shape[:2]
    n = w if direction in ("left", "right") else h
    ramp = np.linspace(1.0 + strength, 1.0 - strength, n)
    if direction == "right":
        ramp = ramp[::-1]
    field = np.tile(ramp, (h, 1)) if direction in ("left", "right") else np.tile(ramp[:, None], (1, w))
    out = img.astype(np.float32) * field[:, :, None]
    return np.clip(out, 0, 255).astype(np.uint8)

def add_noise(img, sigma, rng):
    noise = rng.normal(0, sigma, img.shape)
    return np.clip(img.astype(np.float32) + noise, 0, 255).astype(np.uint8)

def motion_blur(img, k=9, angle=0):
    kern = np.zeros((k, k), np.float32)
    kern[k // 2, :] = 1.0
    M = cv2.getRotationMatrix2D((k / 2.0 - 0.5, k / 2.0 - 0.5), angle, 1.0)
    kern = cv2.warpAffine(kern, M, (k, k))
    kern /= kern.sum()
    return cv2.filter2D(img, -1, kern)

def prepare_background(path, rng):
    bg = cv2.imread(path, cv2.IMREAD_COLOR)
    if bg is None:
        raise IOError("cannot read " + path)
    h, w = bg.shape[:2]
    s = max(SCENE_W / float(w), SCENE_H / float(h))
    bg = cv2.resize(bg, (int(np.ceil(w * s)), int(np.ceil(h * s))), interpolation=cv2.INTER_AREA)
    h, w = bg.shape[:2]
    x = int(rng.integers(0, w - SCENE_W + 1))
    y = int(rng.integers(0, h - SCENE_H + 1))
    return bg[y:y + SCENE_H, x:x + SCENE_W].copy()

# -----
# scene specification
# -----
def scene_plan():
    """Explicit, reproducible description of every scene in the dataset.

    scale is relative to the reference template width of TEMPLATE_W pixels,
    rot is in degrees, tilt is the projective viewpoint change, pos is the
    object centre as a fraction of the scene, occ is the occluded fraction.
    """
    B = ["bg_ug_campus.jpg", "bg_balme_library.jpg", "bg_mensah_sarbah.jpg",
         "bg_accra_street.jpg", "bg_accra_scene.jpg", "bg_office_desk.jpg",
         "bg_brick_wall.jpg", "bg_library_interior.jpg", "bg_lecture_interior.jpg"]

    def inst(scale=1.0, rot=0.0, tilt=(0.0, 0.0), pos=(0.5, 0.5), occ=0.0):
        return {"scale": scale, "rot": rot, "tilt": tilt, "pos": pos, "occ": occ}

    dev = [
        ("dev01_baseline_a", B[0], "baseline", [inst(1.0, 0, (0, 0), (0.30, 0.45))], {}),
        ("dev02_baseline_b", B[6], "baseline", [inst(1.15, 0, (0, 0), (0.62, 0.40))], {}),
        ("dev03_baseline_two", B[7], "baseline", [inst(1.0, 0, (0, 0), (0.25, 0.35)),
                                                inst(1.0, 0, (0, 0), (0.72, 0.62))], {}),
        ("dev04_scale_small", B[1], "scale", [inst(0.60, 0, (0, 0), (0.40, 0.55))], {}),
        ("dev05_scale_large", B[2], "scale", [inst(1.75, 0, (0, 0), (0.55, 0.50))], {}),
        ("dev06_scale_mixed", B[8], "scale", [inst(0.70, 0, (0, 0), (0.22, 0.30)),
                                             inst(1.45, 0, (0, 0), (0.70, 0.60))], {}),
        ("dev07_rot_small", B[3], "rotation", [inst(1.0, 8, (0, 0), (0.45, 0.45))], {}),
        ("dev08_rot_medium", B[4], "rotation", [inst(1.1, 22, (0, 0), (0.55, 0.50))], {}),
        ("dev09_rot_large", B[5], "rotation", [inst(1.0, 45, (0, 0), (0.40, 0.50))], {}),
        ("dev10_illum_dark", B[0], "illumination", [inst(1.05, 0, (0, 0), (0.50, 0.50))],
            {"gamma": 0.45}),
        ("dev11_illum_bright", B[6], "illumination", [inst(1.05, 0, (0, 0), (0.45, 0.45))],
            {"gamma": 1.9}),
        ("dev12_illum_gradient", B[7], "illumination", [inst(1.1, 0, (0, 0), (0.62, 0.50))],
            {"gradient": ("left", 0.55)}),
        ("dev13_view_tilt", B[1], "viewpoint", [inst(1.2, 0, (0.16, 0.05), (0.45, 0.50))], {}),
        ("dev14_view_tilt_rot", B[2], "viewpoint", [inst(1.2, 12, (0.22, 0.08), (0.55, 0.50))], {}),
        ("dev15_occ_light", B[3], "occlusion", [inst(1.1, 0, (0, 0), (0.40, 0.50), 0.18)], {}),
        ("dev16_occ_heavy", B[4], "occlusion", [inst(1.1, 0, (0, 0), (0.55, 0.50), 0.42)], {}),
        ("dev17_clutter", B[5], "clutter", [inst(1.0, 0, (0, 0), (0.30, 0.55))],
            {"distractors": 2}),
        ("dev18_noise", B[8], "noise", [inst(1.1, 0, (0, 0), (0.50, 0.45))], {"noise": 14}),
        ("dev19_blur", B[0], "noise", [inst(1.1, 0, (0, 0), (0.45, 0.55))], {"blur": (9, 20)}),
        ("dev20_negative", B[6], "negative", [], {"distractors": 2}),
    ]
    test = [

```

```

("test01_baseline", B[4], "baseline", [inst(1.05, 0, (0, 0), (0.35, 0.42))], {}),
("test02_baseline_two", B[5], "baseline", [inst(1.0, 0, (0, 0), (0.24, 0.60)),
                                          inst(1.2, 0, (0, 0), (0.70, 0.35))], {}),
("test03_scale_small", B[7], "scale", [inst(0.55, 0, (0, 0), (0.60, 0.40))], {}),
("test04_scale_large", B[3], "scale", [inst(1.90, 0, (0, 0), (0.45, 0.52))], {}),
("test05_rot_small", B[8], "rotation", [inst(1.05, 6, (0, 0), (0.38, 0.48))], {}),
("test06_rot_medium", B[1], "rotation", [inst(1.15, 30, (0, 0), (0.55, 0.45))], {}),
("test07_rot_ninety", B[2], "rotation", [inst(1.0, 90, (0, 0), (0.45, 0.50))], {}),
("test08_illum_dark", B[6], "illumination", [inst(1.1, 0, (0, 0), (0.52, 0.48))],
{"gamma": 0.40}),
("test09_illum_gradient", B[0], "illumination", [inst(1.05, 0, (0, 0), (0.30, 0.50))],
{"gradient": ("right", 0.60)}),
("test10_view_tilt", B[4], "viewpoint", [inst(1.25, 0, (0.20, 0.07), (0.50, 0.50))], {}),
("test11_occ_light", B[5], "occlusion", [inst(1.1, 0, (0, 0), (0.42, 0.45), 0.20)], {}),
("test12_occ_heavy", B[7], "occlusion", [inst(1.15, 0, (0, 0), (0.58, 0.52), 0.45)], {}),
("test13_clutter", B[8], "clutter", [inst(1.05, 0, (0, 0), (0.68, 0.55))],
{"distractors": 3}),
("test14_noise", B[3], "noise", [inst(1.1, 0, (0, 0), (0.47, 0.47))], {"noise": 18}),
("test15_combined", B[1], "combined", [inst(0.80, 15, (0.12, 0.04), (0.35, 0.45), 0.22),
                                       inst(1.30, 0, (0, 0), (0.72, 0.58))],
{"gamma": 0.65, "noise": 8}),
("test16_negative", B[2], "negative", [], {"distractors": 2}),
]
plan = []
for name, bg, cond, insts, photo in dev:
    plan.append((name, bg, "dev", cond, insts, photo))
for name, bg, cond, insts, photo in test:
    plan.append((name, bg, "test", cond, insts, photo))
return plan

def main():
    rng = np.random.default_rng(SEED)
    master = trim_alpha(load_rgba(os.path.join(DATA, "template", "ug_logo_master.png")))
    ref = scale_to_width(master, TEMPLATE_W)

    # The stored template is the emblem composited on a white card, which is
    # how it appears on real printed and web material.
    a = ref[:, :, 3:4].astype(np.float32) / 255.0
    tml = (ref[:, :, :3] * a + 255.0 * (1 - a)).astype(np.uint8)
    cv2.imwrite(os.path.join(DATA, "template", "ug_logo_template.png"), tml)
    cv2.imwrite(os.path.join(DATA, "template", "ug_logo_mask.png"),
                (ref[:, :, 3] & 16).astype(np.uint8) * 255)
    print("template size %dx%d" % (tml.shape[1], tml.shape[0]))

    distractors = [trim_alpha(load_rgba(os.path.join(DATA, "distractors", f)))
                   for f in sorted(os.listdir(os.path.join(DATA, "distractors")))]

    ann = {"dev": {}, "test": {}}
    for name, bgfile, split, cond, insts, photo in scene_plan():
        scene = prepare_background(os.path.join(DATA, "backgrounds", bgfile), rng)
        boxes = []

        for extra in range(photo.get("distractors", 0)):
            d = distractors[extra % len(distractors)]
            d = scale_to_width(d, int(rng.integers(70, 130)))
            d = rotate_rgba(d, float(rng.uniform(-15, 15)))
            dx = int(rng.integers(20, SCENE_W - d.shape[1] - 20))
            dy = int(rng.integers(20, SCENE_H - d.shape[0] - 20))
            alpha_paste(scene, d, dx, dy)

        for spec in insts:
            obj = scale_to_width(master, max(16, int(round(TEMPLATE_W * spec["scale"]))))
            obj = perspective_rgba(obj, spec["tilt"][0], spec["tilt"][1])
            obj = rotate_rgba(obj, spec["rot"])
            oh, ow = obj.shape[:2]
            cx = int(spec["pos"][0] * SCENE_W)
            cy = int(spec["pos"][1] * SCENE_H)
            x = int(np.clip(cx - ow // 2, 4, SCENE_W - ow - 4))
            y = int(np.clip(cy - oh // 2, 4, SCENE_H - oh - 4))
            box = alpha_paste(scene, obj, x, y)
            if box is None:
                continue
            record = {"bbox": box, "scale": spec["scale"], "rotation": spec["rot"],
                    "tilt": list(spec["tilt"]), "occlusion": spec["occ"]}
            if spec["occ"] & 0:
                record["occluded_side"] = occlude(scene, box, spec["occ"], rng)
            boxes.append(record)
        if "gamma" in photo:
            scene = apply_gamma(scene, photo["gamma"])
        if "gradient" in photo:
            d, s = photo["gradient"]
            scene = apply_light_gradient(scene, s, d)
        if "blur" in photo:
            scene = motion_blur(scene, photo["blur"][0], photo["blur"][1])
        if "noise" in photo:
            scene = add_noise(scene, photo["noise"], rng)

        folder = "dev_set" if split == "dev" else "test_set"
        out = os.path.join(DATA, folder, name + ".jpg")
        cv2.imwrite(out, scene, [cv2.IMWRITE_JPEG_QUALITY, 92])
        ann[split][name + ".jpg"] = {
            "condition": cond, "background": bgfile,
            "photometric": {k: v for k, v in photo.items() if k != "distractors"},
            "distractors": photo.get("distractors", 0),
            "objects": boxes,
        }
    print("%-24s %-12s objects=%d" % (name, cond, len(boxes)))

os.makedirs(os.path.join(DATA, "annotations"), exist_ok=True)

```

```

for split, key in (("dev", "dev_annotations.json"), ("test", "test_annotations.json")):
    with open(os.path.join(DATA, "annotations", key), "w", encoding="utf-8") as fh:
        json.dump(ann[split], fh, indent=2)
    n_dev = sum(len(v["objects"]) for v in ann["dev"].values())
    n_test = sum(len(v["objects"]) for v in ann["test"].values())
    print("dev : %d images, %d annotated objects" % (len(ann["dev"]), n_dev))
    print("test : %d images, %d annotated objects" % (len(ann["test"]), n_test))

if __name__ == "__main__":
    main()

```

src/detector.py

```

"""
detector.py
Classical Object Detection and Localisation

Classical detection and localisation of the University of Ghana coat of arms.
No pre-trained recognition network is used anywhere. Everything is built from
classical operators:

    * normalised cross correlation template matching (masked and unmasked)
    * an image pyramid for scale search
    * a rotated template bank for orientation search
    * an explicit sliding window search with configurable window and step
    * greedy non maximum suppression
    * SIFT keypoints, ratio test matching and RANSAC homography
    * HSV colour segmentation used as a region proposal stage

Author: Nii Yartey Gidiglo
"""

import time

import cv2
import numpy as np

# -----
# basic geometry helpers
# -----

def iou(box_a, box_b):
    """Intersection over union of two boxes given as [x, y, w, h]."""
    ax, ay, aw, ah = box_a
    bx, by, bw, bh = box_b
    x0, y0 = max(ax, bx), max(ay, by)
    x1, y1 = min(ax + aw, bx + bw), min(ay + ah, by + bh)
    if x1 <= x0 or y1 <= y0:
        return 0.0
    inter = float((x1 - x0) * (y1 - y0))
    return inter / (aw * ah + bw * bh - inter)

def nms_indices(boxes, scores, iou_thr=0.30):
    """Greedy non maximum suppression, returning the indices that survive.

    Dense correlation produces a cluster of responses around every true
    location, so the raw peaks must be reduced to one detection per object.
    """
    if not boxes:
        return []
    order = list(np.argsort(scores)[::-1])
    keep = []
    suppressed = np.zeros(len(boxes), bool)
    for i in order:
        if suppressed[i]:
            continue
        keep.append(int(i))
        for j in order:
            if not suppressed[j] and j != i and iou(boxes[i], boxes[j]) > iou_thr:
                suppressed[j] = True
    return keep

def non_max_suppression(boxes, scores, iou_thr=0.30):
    keep = nms_indices(boxes, scores, iou_thr)
    return [boxes[i] for i in keep], [scores[i] for i in keep]

# -----
# preprocessing
# -----

def preprocess(bgr, clahe=False, blur=0):
    """Convert to grey scale, optionally equalise locally and denoise.

    Grey scale is used because normalised cross correlation is defined on a
    single channel and because the emblem is discriminated mainly by its
    shape and internal contrast rather than by absolute colour.
    """
    grey = cv2.cvtColor(bgr, cv2.COLOR_BGR2GRAY)
    if blur:
        k = blur if blur % 2 == 1 else blur + 1
        grey = cv2.GaussianBlur(grey, (k, k), 0)
    if clahe:

```

```

        grey = cv2.createCLAHE(clipLimit=2.0, tileGridSize=(8, 8)).apply(grey)
    return grey

# -----
# template bank
# -----
def rotate_with_mask(tmpl, mask, angle):
    """Rotate a template and its mask, expanding the canvas."""
    h, w = tmpl.shape[:2]
    M = cv2.getRotationMatrix2D((w / 2.0, h / 2.0), angle, 1.0)
    cos, sin = abs(M[0, 0]), abs(M[0, 1])
    nw, nh = int(h * sin + w * cos), int(h * cos + w * sin)
    M[0, 2] += nw / 2.0 - w / 2.0
    M[1, 2] += nh / 2.0 - h / 2.0
    rt = cv2.warpAffine(tmpl, M, (nw, nh), flags=cv2.INTER_LINEAR,
                        borderMode=cv2.BORDER_CONSTANT, borderValue=255)
    rm = cv2.warpAffine(mask, M, (nw, nh), flags=cv2.INTER_NEAREST,
                        borderMode=cv2.BORDER_CONSTANT, borderValue=0)
    return rt, rm

def build_template_bank(tmpl, mask, scales=(1.0,), angles=(0.0,), min_side=18):
    """Return a list of (template, mask, scale, angle) at every scale and angle.

    Template matching is invariant to neither scale nor rotation, so the
    invariance has to be bought explicitly by searching over a bank of
    transformed templates. The cost of the search grows linearly with the
    size of the bank, which is the trade off examined in the report.
    """
    bank = []
    for s in scales:
        w = max(min_side, int(round(tmpl.shape[1] * s)))
        h = max(min_side, int(round(tmpl.shape[0] * s)))
        interp = cv2.INTER_AREA if s < 1.0 else cv2.INTER_CUBIC
        ts = cv2.resize(tmpl, (w, h), interpolation=interp)
        ms = cv2.resize(mask, (w, h), interpolation=cv2.INTER_NEAREST)
        for a in angles:
            if abs(a) < 1e-6:
                bank.append((ts, ms, s, 0.0))
            else:
                rt, rm = rotate_with_mask(ts, ms, a)
                bank.append((rt, rm, s, a))
    return bank

# -----
# method 1 and 2: dense template matching over a template bank
# -----

def _peaks_from_response(resp, tw, th, threshold, offset=(0, 0), max_peaks=300):
    """Collect the local maxima of the correlation surface above the threshold.

    Taking every pixel above the threshold would return tens of thousands of
    positions around each response peak. A grey scale dilation keeps only
    positions that are the maximum of their neighbourhood, which is a cheap
    local peak detector and leaves non maximum suppression very little to do.
    """
    resp = np.nan_to_num(resp, nan=-1.0, posinf=-1.0, neginf=-1.0)
    k = max(3, int(min(tw, th) * 0.5) | 1)
    local_max = cv2.dilate(resp, cv2.getStructuringElement(cv2.MORPH_RECT, (k, k)))
    ys, xs = np.where((resp >= threshold) & (resp >= local_max - 1e-6))
    if len(xs) == 0:
        return [], []
    scores = resp[ys, xs]
    order = np.argsort(scores)[::-1][:max_peaks]
    boxes = [int(xs[i]) + offset[0], int(ys[i]) + offset[1], tw, th] for i in order]
    return boxes, [float(scores[i]) for i in order]

def template_match(grey, bank, threshold, masked=True, nms_iou=0.30, rois=None):
    """Dense normalised cross correlation search over a template bank.

    cv2.matchTemplate slides the template over every pixel position, so this
    is itself an exhaustive sliding window search with a step of one pixel,
    implemented with an optimised correlation. The masked variant excludes
    the rectangular corners that do not belong to the emblem, which matters
    because the coat of arms is not a rectangle.
    """
    boxes, scores, meta = [], [], []
    regions = rois if rois else [[0, 0, grey.shape[1], grey.shape[0]]]
    for tmpl, mask, s, a in bank:
        th, tw = tmpl.shape[:2]
        for rx, ry, rw, rh in regions:
            sub = grey[ry:ry + rh, rx:rx + rw]
            if sub.shape[0] < th or sub.shape[1] < tw:
                continue
            if masked:
                resp = cv2.matchTemplate(sub, tmpl, cv2.TM_CCORF_NORMED, mask=mask)
            else:
                resp = cv2.matchTemplate(sub, tmpl, cv2.TM_CCORF_NORMED)
            b, sc = _peaks_from_response(resp, tw, th, threshold, (rx, ry))
            boxes.extend(b)
            scores.extend(sc)
            meta.extend([(s, a)] * len(b))
    if not boxes:
        return [], [], []
    keep = nms_indices(boxes, scores, nms_iou)
    return ([boxes[i] for i in keep], [scores[i] for i in keep],

```

```

        [meta[i] for i in keep])

def response_map(grey, tmpl, mask, masked=True):
    """Single correlation surface, used for the intermediate visualisations."""
    if masked:
        resp = cv2.matchTemplate(grey, tmpl, cv2.TM_CCOEFF_NORMED, mask=mask)
    else:
        resp = cv2.matchTemplate(grey, tmpl, cv2.TM_CCOEFF_NORMED)
    return np.nan_to_num(resp, nan=-1.0, posinf=-1.0, neginf=-1.0)

# -----
# method 3: explicit sliding window search
# -----

def sliding_window_detect(grey, tmpl, mask, win_sizes, step, threshold,
                          nms_iou=0.30):
    """Explicit sliding window detector.

    For every window position the patch is resized to the canonical template
    size and compared with the template using the masked zero mean normalised
    cross correlation


$$NCC = \frac{\sum(w' \cdot t')}{(|w'| \cdot |t'|)}$$


    where w' and t' are the mean removed pixel values inside the mask. This
    is the textbook detector: a search strategy (where to look), a comparison
    function (how similar), and a decision rule (the threshold). Window size,
    step size and threshold are exposed so their effect can be measured.
    """
    th, tw = tmpl.shape[:2]
    m = mask &gt; 0
    t = tmpl.astype(np.float32)[m]
    t = t - t.mean()
    tn = np.linalg.norm(t) + 1e-9

    H, W = grey.shape[:2]
    boxes, scores, windows = [], [], 0
    t0 = time.perf_counter()
    for (wh, ww) in win_sizes:
        if wh &gt; H or ww &gt; W:
            continue
        for y in range(0, H - wh + 1, step):
            for x in range(0, W - ww + 1, step):
                patch = grey[y:y + wh, x:x + ww]
                if (wh, ww) != (th, tw):
                    patch = cv2.resize(patch, (tw, th), interpolation=cv2.INTER_AREA)
                v = patch.astype(np.float32)[m]
                v = v - v.mean()
                ncc = float(np.dot(v, t) / ((np.linalg.norm(v) + 1e-9) * tn))
                windows += 1
                if ncc &gt;= threshold:
                    boxes.append([x, y, ww, wh])
                    scores.append(ncc)
    elapsed = time.perf_counter() - t0
    kept_b, kept_s = non_max_suppression(boxes, scores, nms_iou)
    return kept_b, kept_s, {"windows": windows, "seconds": elapsed}

def window_sizes_from_scales(tmpl_shape, scales):
    th, tw = tmpl_shape[:2]
    out = []
    for s in scales:
        out.append((max(8, int(round(th * s))), max(8, int(round(tw * s)))))
    return out

# -----
# method 4: SIFT feature matching with RANSAC homography
# -----

def feature_match_detect(grey, tmpl_grey, ratio=0.75, min_inliers=8,
                         ransac_thresh=4.0, max_instances=3):
    """Locate the emblem from local features instead of raw intensities.

    SIFT descriptors are invariant to scale and rotation and largely
    invariant to affine illumination change, so this stage recovers the
    instances that fixed template correlation loses. The homography from
    RANSAC gives the localisation: the four template corners are mapped into
    the scene and the axis aligned box around them is reported.

    A single homography can only explain one instance, so the search is
    repeated: after each accepted instance the scene keypoints that fall
    inside the detected quadrilateral are removed and the remaining matches
    are fitted again. This recovers the second emblem in the scenes that
    contain two.
    """
    sift = cv2.SIFT_create()
    kp1, des1 = sift.detectAndCompute(tmpl_grey, None)
    kp2, des2 = sift.detectAndCompute(grey, None)
    info = {"template_keypoints": len(kp1), "scene_keypoints": len(kp2),
            "good_matches": 0, "inliers": 0, "instances": 0, "quads": []}
    if des1 is None or des2 is None or len(kp2) &lt; 2:
        return [], [], info

    matcher = cv2.BFMatcher(cv2.NORM_L2)
    knn = matcher.knnMatch(des1, des2, k=2)

```

```

good = [m for m, n in (p for p in knn if len(p) == 2)
        if m.distance < ratio * n.distance]
info["good_matches"] = len(good)
info["good_match_objects"] = good
info["template_kp"] = kp1
info["scene_kp"] = kp2

h, w = tmpl_grey.shape[:2]
corners = np.float32([[0, 0], [w, 0], [w, h], [0, h]]).reshape(-1, 1, 2)
remaining = list(good)
boxes, scores = [], []
for _ in range(max_instances):
    if len(remaining) < min_inliers:
        break
    src = np.float32([kp1[m.queryIdx].pt for m in remaining]).reshape(-1, 1, 2)
    dst = np.float32([kp2[m.trainIdx].pt for m in remaining]).reshape(-1, 1, 2)
    H, status = cv2.findHomography(src, dst, cv2.RANSAC, ransac_thresh)
    if H is None or status is None:
        break
    inliers = int(status.sum())
    if inliers < min_inliers:
        break
    proj = cv2.perspectiveTransform(corners, H).reshape(-1, 2)
    x0, y0 = float(proj[:, 0].min()), float(proj[:, 1].min())
    bw = float(proj[:, 0].max()) - x0
    bh = float(proj[:, 1].max()) - y0
    keep_outside = [m for k, m in enumerate(remaining) if not status[k]]
    if 10 < bw & bw < grey.shape[1] * 1.5 and 10 < bh & bh < grey.shape[0] * 1.5:
        boxes.append([int(round(x0)), int(round(y0)),
                      int(round(bw)), int(round(bh))])
        scores.append(inliers / float(max(1, len(good))))
        info["quads"].append(proj.tolist())
        info["inliers"] += inliers
    remaining = keep_outside
info["instances"] = len(boxes)
if boxes:
    info["projected_quad"] = info["quads"][0]
return boxes, scores, info

# -----
# method 5: colour segmentation used as a region proposal stage
# -----

def colour_proposals(bgr, pad=0.45, min_area=280, max_regions=12):
    """Propose regions that contain the navy blue of the shield.

    The emblem has a saturated dark blue field. Thresholding the hue and
    saturation channels of HSV, closing the result and taking connected
    components gives a small number of candidate rectangles. Template
    matching then runs only inside those rectangles, which cuts the search
    space by a large factor without changing the comparison function.
    """
    hsv = cv2.cvtColor(bgr, cv2.COLOR_BGR2HSV)
    mask = cv2.inRange(hsv, np.array([100, 70, 25]), np.array([140, 255, 220]))
    kernel = cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (9, 9))
    mask = cv2.morphologyEx(mask, cv2.MORPH_CLOSE, kernel, iterations=2)
    mask = cv2.morphologyEx(mask, cv2.MORPH_OPEN, kernel, iterations=1)
    n, labels, stats, _ = cv2.connectedComponentsWithStats(mask, 8)
    regions = []
    H, W = mask.shape
    for i in range(1, n):
        x, y, w, h, area = stats[i]
        if area < min_area:
            continue
        ar = w / float(h)
        if ar < 0.25 or ar > 4.0:
            continue
        px, py = int(w * pad), int(h * pad)
        rx, ry = max(0, x - px), max(0, y - py)
        rw, rh = min(W - rx, w + 2 * px), min(H - ry, h + 2 * py)
        regions.append([int(rx), int(ry), int(rw), int(rh)], int(area))
    regions.sort(key=lambda r: -r[1])
    return [r[0] for r in regions[:max_regions]], mask

def hybrid_detect(bgr, grey, bank, threshold, nms_iou=0.30):
    """Colour proposals followed by masked template matching inside them."""
    rois, mask = colour_proposals(bgr)
    if not rois:
        return [], [], {"rois": 0, "search_fraction": 0.0, "mask": mask}
    covered = sum(r[2] * r[3] for r in rois)
    boxes, scores, _ = template_match(grey, bank, threshold, True, nms_iou, rois)
    frac = covered / float(grey.shape[0] * grey.shape[1])
    return boxes, scores, {"rois": len(rois), "search_fraction": frac, "mask": mask,
                          "roi_boxes": rois}

# -----
# drawing
# -----

def draw_boxes(img, boxes, scores=None, colour=(0, 200, 0), label=None, thickness=2):
    out = img.copy()
    for i, (x, y, w, h) in enumerate(boxes):
        cv2.rectangle(out, (x, y), (x + w, y + h), colour, thickness)
        text = ""
        if label:

```

```

        text = label
    if scores is not None and i < len(scores):
        text = (text + " " if text else "") + "%.2f" % scores[i]
    if text:
        cv2.rectangle(out, (x, y - 18), (x + 8 * len(text) + 6, y), colour, -1)
        cv2.putText(out, text, (x + 3, y - 5), cv2.FONT_HERSHEY_SIMPLEX, 0.45,
                    (255, 255, 255), 1, cv2.LINE_AA)
return out

```

src/evaluate.py

```

"""
evaluate.py
Classical Object Detection and Localisation

Detection evaluation utilities: greedy matching of detections to ground
truth at a fixed intersection over union, and the standard detection
measures precision, recall and F1 score.

A detection counts as a true positive when its intersection over union with
an unmatched ground truth box is at least IOU_MATCH. Every remaining
detection is a false positive and every unmatched ground truth object is a
false negative, that is a missed detection.
"""

import csv
import json
import os

from detector import iou

IOU_MATCH = 0.5

def match_detections(dets, gts, iou_match=IOU_MATCH):
    """Greedy match detections (highest score first) to ground truth boxes.

    Returns (tp, fp, fn, per_detection_flags, matched_iou).
    """
    used = [False] * len(gts)
    flags, ious = [], []
    tp = 0
    for d in dets:
        best, best_j = 0.0, -1
        for j, g in enumerate(gts):
            if used[j]:
                continue
            v = iou(d, g)
            if v > best:
                best, best_j = v, j
        if best >= iou_match and best_j >= 0:
            used[best_j] = True
            tp += 1
            flags.append(True)
            ious.append(best)
        else:
            flags.append(False)
            ious.append(best)
    fp = len(dets) - tp
    fn = used.count(False)
    return tp, fp, fn, flags, ious

def prf(tp, fp, fn):
    precision = tp / float(tp + fp) if (tp + fp) else 0.0
    recall = tp / float(tp + fn) if (tp + fn) else 0.0
    f1 = 2 * precision * recall / (precision + recall) if (precision + recall) else 0.0
    return precision, recall, f1

def evaluate_split(results, annotations, iou_match=IOU_MATCH):
    """Aggregate over a whole split and also per condition.

    `results` maps image name to a list of detection boxes ordered by score.
    """
    tot = {"tp": 0, "fp": 0, "fn": 0}
    per_cond = {}
    per_image = {}
    all_iou = []
    for name, meta in annotations.items():
        gts = [o["bbox"] for o in meta["objects"]]
        dets = results.get(name, [])
        tp, fp, fn, flags, ious = match_detections(dets, gts, iou_match)
        all_iou.extend([v for v, f in zip(ious, flags) if f])
        tot["tp"] += tp
        tot["fp"] += fp
        tot["fn"] += fn
        c = per_cond.setdefault(meta["condition"], {"tp": 0, "fp": 0, "fn": 0})
        c["tp"] += tp
        c["fp"] += fp
        c["fn"] += fn
        per_image[name] = {"tp": tp, "fp": fp, "fn": fn,
                           "condition": meta["condition"]}
    p, r, f = prf(tot["tp"], tot["fp"], tot["fn"])
    out = {"tp": tot["tp"], "fp": tot["fp"], "fn": tot["fn"],
           "precision": p, "recall": r, "f1": f,
           "mean_iou_of_true_positives": (sum(all_iou) / len(all_iou)) if all_iou else 0.0,
           "per_condition": {}, "per_image": per_image}

```

```

    for cond, c in sorted(per_cond.items()):
        p, r, f = prf(c["tp"], c["fp"], c["fn"])
        out["per_condition"][cond] = {"tp": c["tp"], "fp": c["fp"], "fn": c["fn"],
                                      "precision": p, "recall": r, "f1": f}

    return out

def load_annotations(path):
    with open(path, "r", encoding="utf-8") as fh:
        return json.load(fh)

def write_csv(path, header, rows):
    """Write a table, quoting any field that contains a comma."""
    os.makedirs(os.path.dirname(path), exist_ok=True)
    with open(path, "w", encoding="utf-8", newline="") as fh:
        writer = csv.writer(fh)
        writer.writerow(header)
        for row in rows:
            writer.writerow([str(v) for v in row])

```

src/run_experiments.py

```

"""
run_experiments.py
Classical Object Detection and Localisation

Runs every experiment reported in the write up and saves the tables,
figures and annotated detection images under results/.

python src/run_experiments.py

Experiments
E1 detection threshold sweep (true, false and missed detections)
E2 masked against unmasked normalised cross correlation
E3 size of the scale pyramid against accuracy and run time
E4 rotated template bank against accuracy and run time
E5 explicit sliding window: window size, step size, threshold, cost
E6 SIFT feature matching with RANSAC, ratio and inlier study
E7 colour segmentation proposals combined with template matching
E8 final comparison on the held out test set
E9 qualitative figures and per image detection output

Author: Nii Yartey Gidiglo
"""

import json
import os
import time

import cv2
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
import numpy as np

import detector as D
import evaluate as E

ROOT = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
DATA = os.path.join(ROOT, "data")
RES = os.path.join(ROOT, "results")
FIG = os.path.join(RES, "figures")
TAB = os.path.join(RES, "tables")
DET = os.path.join(RES, "detections")
for d in (RES, FIG, TAB, DET):
    os.makedirs(d, exist_ok=True)

plt.rcParams.update({"figure.dpi": 130, "font.size": 9, "axes.grid": True,
                    "grid.alpha": 0.3, "savefig.bbox": "tight"})

# scale pyramids, expressed as multiples of the reference template size
SCALES_1 = [1.0]
SCALES_5 = [0.6, 0.8, 1.0, 1.35, 1.8]
SCALES_11 = [round(0.5 * 1.15 ** i, 4) for i in range(11)]
ANGLES = [-45, -30, -15, 0, 15, 30, 45, 90]

THRESHOLDS = [round(0.30 + 0.025 * i, 3) for i in range(25)] # 0.300 to 0.900
BASE_THRESHOLD = 0.30 # detections are generated once at this low value
SUMMARY = {}

# -----
# loading
# -----
def load_split(split):
    folder = "dev_set" if split == "dev" else "test_set"
    ann = E.load_annotations(os.path.join(DATA, "annotations", "%s_annotations.json" % split))
    images = {}
    for name in ann:
        bgr = cv2.imread(os.path.join(DATA, folder, name), cv2.IMREAD_COLOR)
        images[name] = {"bgr": bgr, "grey": D.preprocess(bgr)}
    return ann, images

def scored_detections(images, bank, masked=True, threshold=BASE_THRESHOLD):
    """Run the dense matcher once per image and keep boxes with scores."""
    out, total_time = {}, 0.0
    for name, im in images.items():
        t0 = time.perf_counter()

```

```

        boxes, scores, meta = D.template_match(im["grey"], bank, threshold, masked)
        total_time += time.perf_counter() - t0
        out[name] = {"boxes": boxes, "scores": scores, "meta": meta}
    return out, total_time / max(1, len(images))

def at_threshold(scored, thr):
    return {n: [b for b, s in zip(v["boxes"], v["scores"]) if s >= thr]
            for n, v in scored.items()}

def sweep(scored, ann, thresholds=THRESHOLDS):
    rows = []
    for thr in thresholds:
        r = E.evaluate_split(at_threshold(scored, thr), ann)
        rows.append((thr, r["tp"], r["fp"], r["fn"], r["precision"], r["recall"], r["f1"]))
    return rows

def best_row(rows):
    return max(rows, key=lambda r: (r[6], r[4]))

def fmt(rows):
    return [[r[0], r[1], r[2], r[3], round(r[4], 4), round(r[5], 4), round(r[6], 4)]
            for r in rows]

SWEEP_HEADER = ["threshold", "true_positives", "false_positives",
                "missed_detections", "precision", "recall", "f1"]

# -----
def main():
    tpl = cv2.imread(os.path.join(DATA, "template", "ug_logo_template.png"), 0)
    mask = cv2.imread(os.path.join(DATA, "template", "ug_logo_mask.png"), 0)
    tpl_bgr = cv2.imread(os.path.join(DATA, "template", "ug_logo_template.png"))
    print("template %dx%d, mask covers %.1f%% of the rectangle"
          % (tpl.shape[1], tpl.shape[0], 100.0 * (mask > 0).mean()))
    SUMMARY["template"] = {"width": int(tpl.shape[1]), "height": int(tpl.shape[0]),
                          "mask_coverage": float((mask > 0).mean())}

    dev_ann, dev_img = load_split("dev")
    test_ann, test_img = load_split("test")
    SUMMARY["dataset"] = {
        "dev_images": len(dev_ann),
        "dev_objects": sum(len(v["objects"]) for v in dev_ann.values()),
        "test_images": len(test_ann),
        "test_objects": sum(len(v["objects"]) for v in test_ann.values()),
    }

    bank1 = D.build_template_bank(tpl, mask, SCALES_1, [0.0])
    bank5 = D.build_template_bank(tpl, mask, SCALES_5, [0.0])
    bank11 = D.build_template_bank(tpl, mask, SCALES_11, [0.0])
    bank_rot = D.build_template_bank(tpl, mask, SCALES_11, ANGLES)

    # -----
    # figure: template and mask
    # -----
    fig, ax = plt.subplots(1, 3, figsize=(7.2, 2.6))
    ax[0].imshow(cv2.cvtColor(tpl_bgr, cv2.COLOR_BGR2RGB)); ax[0].set_title("template (colour)")
    ax[1].imshow(tpl, cmap="gray"); ax[1].set_title("template (grey scale)")
    ax[2].imshow(mask, cmap="gray"); ax[2].set_title("correlation mask")
    for a in ax:
        a.set_xticks([]); a.set_yticks([]); a.grid(False)
    fig.savefig(os.path.join(FIG, "fig01_template.png")); plt.close(fig)

    # -----
    # figure: dataset overview
    # -----
    show = ["dev01_baseline_a.jpg", "dev04_scale_small.jpg", "dev09_rot_large.jpg",
            "dev10_illum_dark.jpg", "dev13_view_tilt.jpg", "dev16_occ_heavy.jpg",
            "dev17_clutter.jpg", "dev19_blur.jpg", "dev20_negative.jpg"]
    fig, axes = plt.subplots(3, 3, figsize=(9.5, 6.4))
    for a, name in zip(axes.ravel(), show):
        img = dev_img[name]["bgr"].copy()
        for o in dev_ann[name]["objects"]:
            x, y, w, h = o["bbox"]
            cv2.rectangle(img, (x, y), (x + w, y + h), (0, 255, 0), 3)
        a.imshow(cv2.cvtColor(img, cv2.COLOR_BGR2RGB))
        a.set_title("%s\n%s" % (name.replace(".jpg", ""), dev_ann[name]["condition"]), fontsize=7)
        a.set_xticks([]); a.set_yticks([]); a.grid(False)
    fig.savefig(os.path.join(FIG, "fig02_dataset.png")); plt.close(fig)

    # =====
    # E1 threshold sweep, single scale template matching, development set
    # =====
    print("\nE1 threshold sweep")
    dev1, t_dev1 = scored_detections(dev_img, bank1, True)
    rows1 = sweep(dev1, dev_ann)
    E.write_csv(os.path.join(TAB, "e1_threshold_single_scale_dev.csv"), SWEEP_HEADER, fmt(rows1))
    b1 = best_row(rows1)
    print(" best single scale: thr=%.3f F1=%.3f (P=%.3f R=%.3f)" % (b1[0], b1[6], b1[4], b1[5]))

    dev11, t_dev11 = scored_detections(dev_img, bank11, True)
    rows11 = sweep(dev11, dev_ann)
    E.write_csv(os.path.join(TAB, "e1_threshold_multi_scale_dev.csv"), SWEEP_HEADER, fmt(rows11))
    b11 = best_row(rows11)
    print(" best multi scale : thr=%.3f F1=%.3f (P=%.3f R=%.3f)" % (b11[0], b11[6], b11[4], b11[5]))
    a = np.array(rows11, float)

```

```

fig, ax = plt.subplots(1, 2, figsize=(9.4, 3.4))
ax[0].plot(a[:, 0], a[:, 1], "o-", label="true detections")
ax[0].plot(a[:, 0], a[:, 2], "s-", label="false detections")
ax[0].plot(a[:, 0], a[:, 3], "^-", label="missed detections")
ax[0].set_yscale("symlog"); ax[0].set_xlabel("correlation threshold")
ax[0].set_ylabel("count"); ax[0].legend(); ax[0].set_title("effect of the threshold on the counts")
ax[1].plot(a[:, 0], a[:, 4], "o-", label="precision")
ax[1].plot(a[:, 0], a[:, 5], "s-", label="recall")
ax[1].plot(a[:, 0], a[:, 6], "^-", label="F1")
ax[1].axvline(b11[0], color="k", ls="--", lw=1)
ax[1].set_xlabel("correlation threshold"); ax[1].set_ylim(-0.02, 1.05)
ax[1].legend(); ax[1].set_title("precision, recall and F1, multi scale, dev set")
fig.savefig(os.path.join(FIG, "fig03_threshold_sweep.png")); plt.close(fig)

SUMMARY["E1"] = {"single_scale_best": list(b1), "multi_scale_best": list(b11),
                "single_scale_seconds_per_image": t_dev1,
                "multi_scale_seconds_per_image": t_dev11}

# =====
# E2 masked against unmasked correlation
# =====
print("E2 masked against unmasked correlation")
dev11u, t_dev11u = scored_detections(dev_img, bank11, False)
rows11u = sweep(dev11u, dev_ann)
E.write_csv(os.path.join(TAB, "e2_threshold_multi_scale_unmasked_dev.csv"),
            SWEEP_HEADER, fmt(rows11u))
b11u = best_row(rows11u)
print(" unmasked best : thr=%3f F1=%3f" % (b11u[0], b11u[6]))
au = np.array(rows11u, float)
fig, ax = plt.subplots(figsize=(5.2, 3.4))
ax.plot(au[:, 0], au[:, 6], "o-", label="masked correlation")
ax.plot(au[:, 0], au[:, 6], "s-", label="unmasked correlation")
ax.set_xlabel("correlation threshold"); ax.set_ylabel("F1")
ax.set_ylim(-0.02, 1.05); ax.legend(); ax.set_title("effect of the correlation mask")
fig.savefig(os.path.join(FIG, "fig04_mask_effect.png")); plt.close(fig)
E.write_csv(os.path.join(TAB, "e2_mask_comparison.csv"),
            ["variant", "best_threshold", "precision", "recall", "f1", "seconds_per_image"],
            [{"masked", b11[0], round(b11[4], 4), round(b11[5], 4), round(b11[6], 4),
              round(t_dev11, 3)},
             {"unmasked", b11u[0], round(b11u[4], 4), round(b11u[5], 4), round(b11u[6], 4),
              round(t_dev11u, 3)}])
SUMMARY["E2"] = {"masked_best": list(b11), "unmasked_best": list(b11u)}

# =====
# E3 size of the scale pyramid
# =====
print("E3 scale pyramid")
dev5, t_dev5 = scored_detections(dev_img, bank5, True)
rows5 = sweep(dev5, dev_ann)
b5 = best_row(rows5)
e3 = [{"1 scale (reference size only)", len(bank1), b1[0], round(b1[4], 4), round(b1[5], 4),
       round(b1[6], 4), round(t_dev1, 3)},
      ["5 scales (0.6 to 1.8)", len(bank5), b5[0], round(b5[4], 4), round(b5[5], 4),
       round(b5[6], 4), round(t_dev5, 3)],
      ["11 scales (0.50 to 2.02)", len(bank11), b11[0], round(b11[4], 4), round(b11[5], 4),
       round(b11[6], 4), round(t_dev11, 3)]]
E.write_csv(os.path.join(TAB, "e3_scale_pyramid.csv"),
            ["configuration", "templates_in_bank", "best_threshold", "precision",
             "recall", "f1", "seconds_per_image"], e3)

for r in e3:
    print(" %-32s F1=%3f %.2f s/image" % (r[0], r[5], r[6]))
SUMMARY["E3"] = e3

# =====
# E4 rotated template bank
# =====
print("E4 rotated template bank")
devr, t_devr = scored_detections(dev_img, bank_rot, True)
rowsr = sweep(devr, dev_ann)
E.write_csv(os.path.join(TAB, "e4_threshold_rotation_bank_dev.csv"), SWEEP_HEADER, fmt(rowsr))
br = best_row(rowsr)
rot_no = E.evaluate_split(at_threshold(dev11, b11[0]), dev_ann)[ "per_condition" ]
rot_yes = E.evaluate_split(at_threshold(devr, br[0]), dev_ann)[ "per_condition" ]
e4 = []
for label, bank, best, per in (("11 scales without rotation", bank11, b11, rot_no),
                              ("11 scales with 8 angles", bank_rot, br, rot_yes)):
    rc = per.get("rotation", {"recall": 0.0, "f1": 0.0})
    vc = per.get("viewpoint", {"recall": 0.0, "f1": 0.0})
    e4.append([label, len(bank), best[0], round(best[6], 4), round(rc["recall"], 4),
              round(vc["recall"], 4),
              round(t_dev11 if "without rotation" in label else t_devr, 3)])
E.write_csv(os.path.join(TAB, "e4_rotation_bank.csv"),
            ["configuration", "templates_in_bank", "best_threshold", "f1_overall",
             "recall_rotated_images", "recall_viewpoint_images", "seconds_per_image"], e4)

for r in e4:
    print(" %-24s F1=%3f rot recall=%3f %.2f s/image" % (r[0], r[3], r[4], r[6]))
SUMMARY["E4"] = e4

# =====
# E5 explicit sliding window detector
# =====
print("E5 sliding window")
subset = ["dev01_baseline_a.jpg", "dev03_baseline_two.jpg", "dev04_scale_small.jpg",
          "dev11_illum_bright.jpg", "dev20_negative.jpg"]
sub_ann = {k: dev_ann[k] for k in subset}
win_sets = {
    "3 window sizes (0.6 / 1.0 / 1.6)": [0.6, 1.0, 1.6],
    "5 window sizes (0.6 to 1.8)": SCALES_5,
    "1 window size (1.0)": [1.0],
}

```

```

steps = [2, 4, 8, 16, 24]
e5_rows = []
for wname, wscales in win_sets.items():
    sizes = D.window_sizes_from_scales(tmpl.shape, wscales)
    for step in steps:
        per_image, windows, secs = {}, 0, 0.0
        raw = {}
        for name in subset:
            b, s, info = D.sliding_window_detect(
                dev_img[name]["grey"], tmpl, mask, sizes, step, 0.50)
            raw[name] = (b, s)
            windows += info["windows"]
            secs += info["seconds"]
        best = None
        for thr in [0.50, 0.55, 0.60, 0.65, 0.70, 0.75]:
            res = {n: [bb for bb, ss in zip(*raw[n]) if ss >= thr] for n in subset}
            r = E.evaluate_split(res, sub_ann)
            if best is None or r["f1"] > best[1]["f1"]:
                best = (thr, r)
        thr, r = best
        e5_rows.append((wname, len(sizes), step, windows, round(secs / len(subset), 3),
            thr, r["tp"], r["fp"], r["fn"], round(r["precision"], 4),
            round(r["recall"], 4), round(r["f1"], 4)))
        print(" %s %24s step=%2d windows=%7d %2f s/image F1=%3f"
            % (wname, step, windows, secs / len(subset), r["f1"]))
E.write_csv(os.path.join(TAB, "e5_sliding_window.csv"),
    ["window_set", "window_sizes", "step_px", "windows_evaluated",
    "seconds_per_image", "best_threshold", "true_positives",
    "false_positives", "missed_detections", "precision", "recall", "f1"], e5_rows)

fig, ax = plt.subplots(1, 2, figsize=(9.4, 3.4))
for wname in win_sets:
    r = [x for x in e5_rows if x[0] == wname]
    ax[0].plot([x[2] for x in r], [x[4] for x in r], "o-", label=wname)
    ax[1].plot([x[2] for x in r], [x[11] for x in r], "o-", label=wname)
    ax[0].set_xlabel("step size in pixels"); ax[0].set_ylabel("seconds per image")
    ax[0].set_yscale("log"); ax[0].set_title("cost of the sliding window search"); ax[0].legend(fontsize=7)
    ax[1].set_xlabel("step size in pixels"); ax[1].set_ylabel("F1")
    ax[1].set_ylim(-0.02, 1.05); ax[1].set_title("accuracy against step size"); ax[1].legend(fontsize=7)
fig.savefig(os.path.join(FIG, "fig05_sliding_window.png")); plt.close(fig)
SUMMARY["E5"] = e5_rows

# sliding window threshold study at a fixed geometry
sizes = D.window_sizes_from_scales(tmpl.shape, [0.6, 1.0, 1.6])
raw = {}
for name in subset:
    b, s, info = D.sliding_window_detect(dev_img[name]["grey"], tmpl, mask, sizes, 4, 0.40)
    raw[name] = (b, s)
rows_sw = []
for thr in [0.40, 0.45, 0.50, 0.55, 0.60, 0.65, 0.70, 0.75, 0.80]:
    res = {n: [bb for bb, ss in zip(*raw[n]) if ss >= thr] for n in subset}
    r = E.evaluate_split(res, sub_ann)
    rows_sw.append([thr, r["tp"], r["fp"], r["fn"], round(r["precision"], 4),
        round(r["recall"], 4), round(r["f1"], 4)])
E.write_csv(os.path.join(TAB, "e5b_sliding_window_threshold.csv"), SWEEP_HEADER, rows_sw)

# =====
# E6 SIFT feature matching with RANSAC
# =====
print("E6 feature matching")
e6_rows, best_fm = [], None
for ratio in [0.70, 0.75, 0.80]:
    for min_inl in [8, 10, 12, 15, 20]:
        res, secs = {}, 0.0
        for name, im in dev_img.items():
            t0 = time.perf_counter()
            b, s, info = D.feature_match_detect(im["grey"], tmpl, ratio=ratio,
                min_inliers=min_inl)
            secs += time.perf_counter() - t0
            res[name] = b
        r = E.evaluate_split(res, dev_ann)
        e6_rows.append([ratio, min_inl, r["tp"], r["fp"], r["fn"], round(r["precision"], 4),
            round(r["recall"], 4), round(r["f1"], 4),
            round(secs / len(dev_img), 3)])
        if best_fm is None or r["f1"] > best_fm[2]:
            best_fm = (ratio, min_inl, r["f1"])
E.write_csv(os.path.join(TAB, "e6_feature_matching.csv"),
    ["lowe_ratio", "min_inliers", "true_positives", "false_positives",
    "missed_detections", "precision", "recall", "f1", "seconds_per_image"], e6_rows)
print(" best feature configuration: ratio=%2f min_inliers=%d F1=%3f % best_fm)
SUMMARY["E6"] = {"best": list(best_fm), "rows": e6_rows}

# =====
# E7 colour proposals combined with template matching
# =====
print("E7 colour proposals and template matching")
hyb, secs, fracs, roi_counts = {}, 0.0, [], []
for name, im in dev_img.items():
    t0 = time.perf_counter()
    b, s, info = D.hybrid_detect(im["bgr"], im["grey"], bank11, BASE_THRESHOLD)
    secs += time.perf_counter() - t0
    hyb[name] = {"boxes": b, "scores": s}
    fracs.append(info["search_fraction"])
    roi_counts.append(info["rois"])
rows7 = sweep(hyb, dev_ann)
E.write_csv(os.path.join(TAB, "e7_threshold_hybrid_dev.csv"), SWEEP_HEADER, fmt(rows7))
b7 = best_row(rows7)
e7 = [{"multi scale template matching over the whole image", round(t_dev11, 3), 1.0,
    b11[0], round(b11[4], 4), round(b11[5], 4), round(b11[6], 4)},
    ["colour proposals then template matching", round(secs / len(dev_img), 3),

```

```

        round(float(np.mean(fracs)), 4), b7[0], round(b7[4], 4), round(b7[5], 4),
        round(b7[6], 4)])
E.write_csv(os.path.join(TAB, "e7_hybrid.csv"),
            ["configuration", "seconds_per_image", "fraction_of_image_searched",
            "best_threshold", "precision", "recall", "f1"], e7)
print(" proposals cover %.1f%% of the image, %.2f s/image, F1=%.3f"
      % (100 * np.mean(fracs), secs / len(dev_img), b7[6]))
SUMMARY["E7"] = {"rows": e7, "mean_rois": float(np.mean(roi_counts))}

# =====
# E8 final evaluation on the held out test set
# =====
print("E8 held out test set")
methods = {}

t1, s1 = scored_detections(test_img, bank1, True)
methods["single scale template matching"] = (at_threshold(t1, b1[0]), s1, b1[0])

t11, s11 = scored_detections(test_img, bank11, True)
methods["multi scale template matching"] = (at_threshold(t11, b11[0]), s11, b11[0])

tr, sr = scored_detections(test_img, bank_rot, True)
methods["multi scale and rotation bank"] = (at_threshold(tr, br[0]), sr, br[0])

sizes = D.window_sizes_from_scales(tmpl.shape, [0.6, 1.0, 1.6])
sw_res, sw_secs = {}, 0.0
for name, im in test_img.items():
    b, s, info = D.sliding_window_detect(im["grey"], tmpl, mask, sizes, 4, 0.55)
    sw_res[name] = b
    sw_secs += info["seconds"]
methods["explicit sliding window (step 4)"] = (sw_res, sw_secs / len(test_img), 0.55)

fm_res, fm_secs, fm_info = {}, 0.0, {}
for name, im in test_img.items():
    t0 = time.perf_counter()
    b, s, info = D.feature_match_detect(im["grey"], tmpl, ratio=best_fm[0],
                                       min_inliers=best_fm[1])
    fm_secs += time.perf_counter() - t0
    fm_res[name] = b
    fm_info[name] = {"boxes": b, "scores": s, "info": info}
methods["SIFT matching with RANSAC"] = (fm_res, fm_secs / len(test_img), best_fm[1])

hy_res, hy_secs = {}, 0.0
for name, im in test_img.items():
    t0 = time.perf_counter()
    b, s, info = D.hybrid_detect(im["bgr"], im["grey"], bank11, b7[0])
    hy_secs += time.perf_counter() - t0
    hy_res[name] = b
methods["colour proposals and template matching"] = (hy_res, hy_secs / len(test_img), b7[0])

# combination: colour guided template matching plus feature verification
comb_res, comb_secs = {}, 0.0
for name, im in test_img.items():
    t0 = time.perf_counter()
    b, s, info = D.hybrid_detect(im["bgr"], im["grey"], bank11, b7[0])
    fb = fm_info[name]["boxes"]
    merged_b = list(b) + list(fb)
    merged_s = list(s) + [0.6 + 0.4 * x for x in fm_info[name]["scores"]]
    keep = D.nms_indices(merged_b, merged_s, 0.30)
    comb_res[name] = [merged_b[i] for i in keep]
    comb_secs += time.perf_counter() - t0
methods["combined colour and template and features"] = (
    comb_res, comb_secs / len(test_img) + fm_secs / len(test_img), None)

e8_rows, per_cond_table = [], {}
for label, (res, secs, thr) in methods.items():
    r = E.evaluate_split(res, test_ann)
    e8_rows.append([label, thr if thr is not None else "n/a", r["tp"], r["fp"], r["fn"],
                    round(r["precision"], 4), round(r["recall"], 4), round(r["f1"], 4),
                    round(r["mean_iou_of_true_positives"], 4), round(secs, 3)])
    per_cond_table[label] = r["per_condition"]
    print(" %-42s P=%.3f R=%.3f F1=%.3f %.2f s/image"
          % (label, r["precision"], r["recall"], r["f1"], secs))
E.write_csv(os.path.join(TAB, "e8_test_set_comparison.csv"),
            ["method", "operating_threshold", "true_positives", "false_positives",
            "missed_detections", "precision", "recall", "f1",
            "mean_iou_of_true_positives", "seconds_per_image"], e8_rows)
SUMMARY["E8"] = e8_rows

conds = sorted({c for v in per_cond_table.values() for c in v if c != "negative"})
rows_pc = []
for label, per in per_cond_table.items():
    rows_pc.append([label] + [round(per.get(c, {"f1": 0.0})["f1"], 3) for c in conds])
E.write_csv(os.path.join(TAB, "e8b_per_condition_f1.csv"), ["method"] + conds, rows_pc)

fig, ax = plt.subplots(figsize=(9.6, 3.8))
width = 0.8 / len(rows_pc)
xs = np.arange(len(conds))
for i, row in enumerate(rows_pc):
    ax.bar(xs + i * width, row[1:], width, label=row[0])
ax.set_xticks(xs + 0.4 - width / 2)
ax.set_xticklabels(conds, rotation=20, ha="right")
ax.set_ylabel("F1"); ax.set_ylim(0, 1.05)
ax.set_title("F1 by imaging condition, held out test set")
ax.legend(fontsize=6.5, ncol=2)
fig.savefig(os.path.join(FIG, "fig08_per_condition.png")); plt.close(fig)

# =====
# E9 qualitative material
# =====

```

```

print("E9 figures and per image output")

# pipeline stages on one image
name = "dev01_baseline_a.jpg"
grey = dev_img[name]["grey"]
resp = D.response_map(grey, tmpl, mask, True)
raw_boxes, raw_scores = D.peaks_from_response(resp, tmpl.shape[1], tmpl.shape[0], 0.45)
kept_b, kept_s = D.non_max_suppression(raw_boxes, raw_scores, 0.30)
stage_pre = D.draw_boxes(dev_img[name]["bgr"], raw_boxes[:60], None, (0, 165, 255), None, 1)
stage_post = D.draw_boxes(dev_img[name]["bgr"], kept_b, kept_s, (0, 220, 0), "logo")
fig, ax = plt.subplots(2, 3, figsize=(10.5, 5.4))
ax[0][0].imshow(cv2.cvtColor(dev_img[name]["bgr"], cv2.COLOR_BGR2RGB)); ax[0][0].set_title("1. input scene")
ax[0][1].imshow(grey, cmap="gray"); ax[0][1].set_title("2. grey scale input")
im2 = ax[0][2].imshow(resp, cmap="inferno"); ax[0][2].set_title("3. correlation surface")
fig.colorbar(im2, ax=ax[0][2], fraction=0.035)
ax[1][0].imshow(resp &gt;= 0.45, cmap="gray"); ax[1][0].set_title("4. surface above the threshold")
ax[1][1].imshow(cv2.cvtColor(stage_pre, cv2.COLOR_BGR2RGB)); ax[1][1].set_title("5. candidates before suppress")
ax[1][2].imshow(cv2.cvtColor(stage_post, cv2.COLOR_BGR2RGB)); ax[1][2].set_title("6. final localisation")
for a in ax.ravel():
    a.set_xticks([]); a.set_yticks([]); a.grid(False)
fig.savefig(os.path.join(FIG, "fig06_pipeline_stages.png")); plt.close(fig)

# colour proposal figure
name = "dev17_clutter.jpg"
rois, cmask = D.colour_proposals(dev_img[name]["bgr"])
vis = dev_img[name]["bgr"].copy()
for x, y, w, h in rois:
    cv2.rectangle(vis, (x, y), (x + w, y + h), (255, 160, 0), 2)
fig, ax = plt.subplots(1, 3, figsize=(10.5, 2.8))
ax[0].imshow(cv2.cvtColor(dev_img[name]["bgr"], cv2.COLOR_BGR2RGB)); ax[0].set_title("scene with clutter")
ax[1].imshow(cmask, cmap="gray"); ax[1].set_title("HSV blue mask after morphology")
ax[2].imshow(cv2.cvtColor(vis, cv2.COLOR_BGR2RGB)); ax[2].set_title("region proposals")
for a in ax:
    a.set_xticks([]); a.set_yticks([]); a.grid(False)
fig.savefig(os.path.join(FIG, "fig07_colour_proposals.png")); plt.close(fig)

# SIFT match figure with RANSAC inliers
name = "test06_rot_medium.jpg"
g = test_img[name]["grey"]
b, s, info = D.feature_match_detect(g, tmpl, ratio=best_fm[0], min_inliers=best_fm[1])
good = info.get("good_match_objects", [])
vis = cv2.drawMatches(tmpl, info["template_kp"], test_img[name]["bgr"], info["scene_kp"],
    good[:60], None,
    matchColor=(0, 220, 0), singlePointColor=(200, 200, 200),
    flags=cv2.DrawMatchesFlags_NOT_DRAW_SINGLE_POINTS)
fig, ax = plt.subplots(figsize=(10.5, 3.4))
ax.imshow(cv2.cvtColor(vis, cv2.COLOR_BGR2RGB))
ax.set_title("SIFT correspondences kept by the ratio test, %s" % name.replace(".jpg", ""))
ax.set_xticks([]); ax.set_yticks([]); ax.grid(False)
fig.savefig(os.path.join(FIG, "fig09_sift_matches.png")); plt.close(fig)

# per image detection output for the whole test set, best method
best_label = max(e8_rows, key=lambda r: r[7])[0]
best_res = methods[best_label][0]
for name, im in test_img.items():
    vis = im["bgr"].copy()
    for o in test_ann[name]["objects"]:
        x, y, w, h = o["bbox"]
        cv2.rectangle(vis, (x, y), (x + w, y + h), (0, 255, 255), 2)
    vis = D.draw_boxes(vis, best_res.get(name, []), None, (0, 200, 0), "UG logo")
    cv2.imwrite(os.path.join(DET, name), vis)
print(" best method on the test set: %s" % best_label)

# success and failure montage
def montage(names, results, path, title):
    cols = 3
    rows = int(np.ceil(len(names) / cols))
    fig, axes = plt.subplots(rows, cols, figsize=(3.4 * cols, 2.4 * rows))
    for a, nm in zip(np.ravel(axes), names):
        vis = test_img[nm]["bgr"].copy()
        for o in test_ann[nm]["objects"]:
            x, y, w, h = o["bbox"]
            cv2.rectangle(vis, (x, y), (x + w, y + h), (0, 255, 255), 2)
        vis = D.draw_boxes(vis, results.get(nm, []), None, (0, 200, 0), None)
        a.imshow(cv2.cvtColor(vis, cv2.COLOR_BGR2RGB))
        a.set_title("%s (%s)" % (nm.replace(".jpg", ""), test_ann[nm]["condition"]), fontsize=7)
    for a in np.ravel(axes):
        a.set_xticks([]); a.set_yticks([]); a.grid(False)
    fig.suptitle(title, fontsize=10)
    fig.savefig(path); plt.close(fig)

tm_res = methods["multi scale template matching"][0]
per_img = E.evaluate_split(tm_res, test_ann["per_image"])
fails = [n for n, v in per_img.items() if v["fn"] &gt; 0 or v["fp"] &gt; 0][:6]
wins = [n for n, v in per_img.items() if v["fn"] == 0 and v["fp"] == 0][:6]
if wins:
    montage(wins, tm_res, os.path.join(FIG, "fig10_success.png"),
        "multi scale template matching: correct cases (yellow is ground truth)")
if fails:
    montage(fails, tm_res, os.path.join(FIG, "fig11_failure.png"),
        "multi scale template matching: failure cases (yellow is ground truth)")

with open(os.path.join(RES, "summary.json"), "w", encoding="utf-8") as fh:
    json.dump(SUMMARY, fh, indent=2, default=str)
print("\nall results written to results/")

if __name__ == "__main__":
    main()

```

src/make_report.py

```
"""
make_report.py
Classical Object Detection and Localisation

Builds the submission PDF from the measured results in results/. Every number
quoted in the report is read from the CSV files written by run_experiments.py,
so the document cannot drift away from the experiments.

python src/make_report.py

Author: Nii Yartey Gidiglo
"""

import csv
import json
import os

from reportlab.lib import colors
from reportlab.lib.enums import TA_CENTER, TA_JUSTIFY
from reportlab.lib.pagesizes import A4
from reportlab.lib.styles import ParagraphStyle, getSampleStyleSheet
from reportlab.lib.units import cm
from reportlab.platypus import (BaseDocTemplate, Frame, Image, KeepTogether,
                                NextPageTemplate, PageBreak, PageTemplate,
                                Paragraph, Preformatted, Spacer, Table,
                                TableStyle)

from reportlab.lib.utils import ImageReader

ROOT = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
RES = os.path.join(ROOT, "results")
FIG = os.path.join(RES, "figures")
TAB = os.path.join(RES, "tables")
OUT = os.path.join(ROOT, "report",
                    "Classical_Object_Detection_and_Localisation.pdf")

NAME = "Nii Yartey Gidiglo"
PROJECT = "Classical object detection and localisation"

PAGE_W, PAGE_H = A4
MARGIN = 2.0 * cm
BODY_W = PAGE_W - 2 * MARGIN

ss = getSampleStyleSheet()
BODY = ParagraphStyle("body", parent=ss["BodyText"], fontName="Helvetica",
                      fontSize=9.6, leading=13.6, alignment=TA_JUSTIFY,
                      spaceAfter=6)
H1 = ParagraphStyle("h1", parent=ss["Heading1"], fontName="Helvetica-Bold",
                    fontSize=14, leading=17, spaceBefore=14, spaceAfter=7,
                    textColor=colors.HexColor("#12305c"))
H2 = ParagraphStyle("h2", parent=ss["Heading2"], fontName="Helvetica-Bold",
                    fontSize=11.2, leading=14, spaceBefore=10, spaceAfter=5,
                    textColor=colors.HexColor("#1d4d8f"))
H3 = ParagraphStyle("h3", parent=ss["Heading3"], fontName="Helvetica-Bold",
                    fontSize=10, leading=13, spaceBefore=8, spaceAfter=4)
CAP = ParagraphStyle("cap", parent=BODY, fontSize=8.2, leading=10.5,
                    alignment=TA_CENTER, textColor=colors.HexColor("#444444"),
                    spaceBefore=3, spaceAfter=10)
EQ = ParagraphStyle("eq", parent=BODY, fontName="Courier", fontSize=9,
                    leading=12.5, alignment=TA_CENTER, spaceBefore=5,
                    spaceAfter=7)
BULLET = ParagraphStyle("bullet", parent=BODY, leftIndent=14, bulletIndent=4,
                        spaceAfter=3)
CODE = ParagraphStyle("code", parent=ss["Code"], fontName="Courier",
                      fontSize=6.0, leading=7.2, spaceAfter=0, spaceBefore=0)
TITLE = ParagraphStyle("title", parent=BODY, fontName="Helvetica-Bold",
                       fontSize=19, leading=24, alignment=TA_CENTER,
                       spaceAfter=8, textColor=colors.HexColor("#12305c"))
SUB = ParagraphStyle("sub", parent=BODY, fontSize=11.5, leading=16,
                    alignment=TA_CENTER, spaceAfter=4)

story = []
FIGN = [0]
TABN = [0]

# -----
# small helpers
# -----
def p(text, style=BODY):
    story.append(Paragraph(text, style))

def h1(text):
    story.append(Paragraph(text, H1))

def h2(text):
    story.append(Paragraph(text, H2))

def h3(text):
    story.append(Paragraph(text, H3))
def bullets(items):
    for it in items:
        story.append(Paragraph(it, BULLET, bulletText="•"))
    story.append(Spacer(1, 4))
def eq(text):
    story.append(Paragraph(text, EQ))
```

```

def figure(fname, caption, width=BODY_W):
    path = os.path.join(FIG, fname)
    if not os.path.exists(path):
        return
    iw, ih = ImageReader(path).getSize()
    w = min(width, BODY_W)
    h = w * ih / float(iw)
    max_h = 15.5 * cm
    if h > max_h:
        h = max_h
        w = h * iw / float(ih)
    FIGN[0] += 1
    story.append(KeepTogether([Image(path, w, h),
                               Paragraph("Figure %d. %s" % (FIGN[0], caption), CAP)]))

def read_table(name):
    with open(os.path.join(TAB, name), "r", encoding="utf-8") as fh:
        rows = list(csv.reader(fh))
    return rows[0], rows[1:]

def table(header, rows, caption, widths=None, fontsize=7.4, align_left_first=True):
    TABN[0] += 1
    data = [[Paragraph("<b>%s</b>" % h.replace(" ", "&nbsp;"),
                      ParagraphStyle("th", parent=BODY, fontSize=fontsize,
                                     leading=fontsize + 2, alignment=TA_CENTER,
                                     textColor=colors.white, spaceAfter=0))
             for h in header]]
    cell = ParagraphStyle("td", parent=BODY, fontSize=fontsize,
                          leading=fontsize + 2, alignment=TA_CENTER, spaceAfter=0)
    cell_l = ParagraphStyle("tdl", parent=cell, alignment=0)
    for r in rows:
        data.append([Paragraph(str(v), cell_l if (i == 0 and align_left_first) else cell)
                     for i, v in enumerate(r)])
    t = Table(data, colWidths=widths, repeatRows=1, hAlign="CENTER")
    t.setStyle(TableStyle([
        ("BACKGROUND", (0, 0), (-1, 0), colors.HexColor("#1d4d8f")),
        ("GRID", (0, 0), (-1, -1), 0.4, colors.HexColor("#9aa7b5")),
        ("VALIGN", (0, 0), (-1, -1), "MIDDLE"),
        ("ROWBACKGROUNDS", (0, 1), (-1, -1),
         [colors.white, colors.HexColor("#eef2f7")]),
        ("LEFTPADDING", (0, 0), (-1, -1), 3),
        ("RIGHTPADDING", (0, 0), (-1, -1), 3),
        ("TOPPADDING", (0, 0), (-1, -1), 2.5),
        ("BOTTOMPADDING", (0, 0), (-1, -1), 2.5),
    ]))
    story.append(KeepTogether([t, Paragraph("Table %d. %s" % (TABN[0], caption), CAP)]))

def num(x, nd=3):
    try:
        return ("%." + str(nd) + "f") % float(x)
    except (TypeError, ValueError):
        return str(x)

def esc(text):
    return (text.replace("&", "&amp;").replace("<", "&lt;").replace(">", "&gt;"))

# -----
# page furniture
# -----
def decorate(canvas, doc):
    canvas.saveState()
    canvas.setFont("Helvetica", 7.5)
    canvas.setFillColor(colors.HexColor("#5a6673"))
    canvas.drawString(MARGIN, 1.15 * cm, PROJECT)
    canvas.drawRightString(PAGE_W - MARGIN, 1.15 * cm,
                          "%s, page %d" % (NAME, doc.page))
    canvas.setStrokeColor(colors.HexColor("#c7d0da"))
    canvas.line(MARGIN, 1.45 * cm, PAGE_W - MARGIN, 1.45 * cm)
    canvas.restoreState()

def blank(canvas, doc):
    pass
def build():
    doc = BaseDocTemplate(OUT, pagesize=A4, leftMargin=MARGIN, rightMargin=MARGIN,
                        topMargin=1.7 * cm, bottomMargin=2.0 * cm,
                        title="Classical Object Detection and Localisation",
                        author=NAME)
    frame = Frame(MARGIN, 2.0 * cm, BODY_W, PAGE_H - 3.7 * cm, id="body")
    doc.addPageTemplates([PageTemplate(id="cover", frames=[frame], onPage=blank),
                          PageTemplate(id="main", frames=[frame], onPage=decorate)])
    doc.build(story)

# -----
# content
# -----
def main():
    summary = json.load(open(os.path.join(RES, "summary.json"), encoding="utf-8"))
    sources = json.load(open(os.path.join(ROOT, "data", "sources.json"), encoding="utf-8"))

    h_e1s, r_e1s = read_table("e1_threshold_single_scale_dev.csv")
    h_e1m, r_e1m = read_table("e1_threshold_multi_scale_dev.csv")
    h_e2, r_e2 = read_table("e2_mask_comparison.csv")
    h_e3, r_e3 = read_table("e3_scale_pyramid.csv")

```

```

h_e4, r_e4 = read_table("e4_rotation_bank.csv")
h_e5, r_e5 = read_table("e5_sliding_window.csv")
h_e5b, r_e5b = read_table("e5b_sliding_window_threshold.csv")
h_e6, r_e6 = read_table("e6_feature_matching.csv")
h_e7, r_e7 = read_table("e7_hybrid.csv")
h_e8, r_e8 = read_table("e8_test_set_comparison.csv")
h_e8b, r_e8b = read_table("e8b_per_condition_f1.csv")

ds = summary["dataset"]
tm_best = max(r_e1m, key=lambda r: float(r[6]))
ss_best = max(r_e1s, key=lambda r: float(r[6]))
e8_best = max(r_e8, key=lambda r: float(r[7]))
e6_best = max(r_e6, key=lambda r: float(r[7]))
sw_row = [r for r in r_e8 if r[0].startswith("explicit sliding")][0]
sw_fine = [r for r in r_e5 if r[0].startswith("3 window sizes") and r[2] == "2"]
sw_coarse = [r for r in r_e5 if r[0].startswith("3 window sizes") and r[2] == "24"]
sw_fine = sw_fine[0] if sw_fine else r_e5[0]
sw_coarse = sw_coarse[0] if sw_coarse else r_e5[-1]

# ----- cover -----
story.append(Spacer(1, 2.4 * cm))
p("COMPUTER VISION PROJECT", SUB)
story.append(Spacer(1, 1.1 * cm))
p("Classical Object Detection and Localisation of the University of Ghana Coat of Arms",
TITLE)
story.append(Spacer(1, 0.7 * cm))
p("Template matching, sliding window search, feature matching and colour "
"segmentation, evaluated on an annotated test set", SUB)
story.append(Spacer(1, 1.5 * cm))
info = Table(["Author", NAME],
[["Repository", "github.com/Qwamzz/Computer-Vision-Projects"],
["Target object", "University of Ghana coat of arms"],
["Evaluation set", "%d annotated scenes, %d object instances"
% (ds["dev_images"] + ds["test_images"],
ds["dev_objects"] + ds["test_objects"])],
["Implementation", "Python 3 and OpenCV, no pre-trained recognition model"]],
colWidths=[4.2 * cm, 10.5 * cm])
info.setStyle(TableStyle([
("GRID", (0, 0), (-1, -1), 0.4, colors.HexColor("#9aa7b5")),
("BACKGROUND", (0, 0), (0, -1), colors.HexColor("#eef2f7")),
("FONTNAME", (0, 0), (0, -1), "Helvetica-Bold"),
("FONTSIZE", (0, 0), (-1, -1), 9),
("TOPPADDING", (0, 0), (-1, -1), 5),
("BOTTPADDING", (0, 0), (-1, -1), 5),
]))
info.hAlign = "CENTER"
story.append(info)
story.append(Spacer(1, 1.4 * cm))
p("<b>Statement on the imagery</b> Every image used in this project is a "
"real photograph or a real vector emblem obtained from Wikimedia Commons "
"under a free licence, and the licence and author of each file are listed "
"in Section 2.2 and in data/SOURCES.md. No image was produced by an image "
"generation model.", ParagraphStyle("note", parent=BODY, fontSize=8.6,
leading=12,
borderPadding=6,
backColor=colors.HexColor("#f4f7fb")))
story.append(NextPageTemplate("main"))
story.append(PageBreak())

# ----- 1 problem -----
h1("1. Problem definition and objectives")
p("The task is to build a system that finds a chosen object category in an "
"image and reports where it is, using classical detection techniques rather "
"than a pre-trained recognition network. The "
"object chosen here is the <b>University of Ghana coat of arms</b>, the "
"emblem that carries three stylised palm trees over a navy shield above the "
"motto <i>Integri Procedamus</i>. It is a good subject for a classical "
"detector: it is rigid, it is planar, it has a fixed internal layout and it "
"has strong internal contrast, which is exactly the situation in which "
"correlation based matching is defensible. It is also a realistic object to "
"detect, since it appears on university signage, letterheads, certificates, "
"identity cards and merchandise, and automatic detection supports document "
"verification and brand monitoring.")
p("The specific objectives are the following.")
bullets([
"Build a template of the object and a test set in which the object appears "
"at different positions, sizes, orientations, illumination levels, "
"backgrounds and degrees of partial occlusion.",
"Implement template matching by normalised cross correlation and measure "
"how the decision threshold trades true detections against false and "
"missed detections.",
"Extend the detector with an explicit sliding window search and measure the "
"effect of window size, step size and threshold on both accuracy and "
"computational cost.",
"Add feature matching with RANSAC and a colour segmentation stage where "
"these improve reliability, and draw a bounding box on every detected "
"instance.",
"Evaluate on a manually specified annotated test set that was not used "
"during development, reporting precision, recall and F1 score.",
"Discuss the conditions under which template matching and sliding window "
"detection become unreliable and explain why.",
])
p("The complete detection progression is implemented and measured end to end:")
eq("input image -&gt; search strategy -&gt; template comparison -&gt; "
"detection decision -&gt; object localisation -&gt; performance evaluation")

# ----- 2 dataset -----
h1("2. Dataset")
h2("2.1 Target object and template")
p("The template is the official University of Ghana coat of arms, taken from ")

```

```

"the public domain vector file on Wikimedia Commons and rendered at a "
"reference width of %d pixels, giving a template of %d by %d pixels. A "
"binary mask of the emblem is stored beside it. The mask matters because "
"the coat of arms is not rectangular: it is a shield with a curved lower "
"edge and a scroll, and it covers only %.1f per cent of its bounding "
"rectangle. The remaining %.1f per cent of the rectangle is background that "
"changes from photograph to photograph, so including it in the comparison "
"adds noise to the similarity score. Section 5.2 measures exactly how much "
"this matters."
% (summary["template"]["width"], summary["template"]["width"],
summary["template"]["height"], 100 * summary["template"]["mask_coverage"],
100 * (1 - summary["template"]["mask_coverage"]))
figure("fig01_template.png",
"The template used throughout: the colour emblem, its grey scale form "
"which is what correlation actually sees, and the binary mask that "
"restricts the comparison to the emblem itself.", 13 * cm)
h2("2.2 Source imagery")
p("All source material was downloaded from Wikimedia Commons by "
"src/download_data.py, which also records the licence and author of each "
"file. Nine real photographs supply the backgrounds and cover outdoor "
"campus scenes, an Accra street scene, an office desk, a brick wall and two "
"interior scenes. Two further university emblems that resemble the target "
"in colour and layout are used as clutter and as hard negatives, so that "
"the false positive rate is not measured only against easy background.")
table(["local file", "source file on Wikimedia Commons", "licence", "author"],
[[s["file"], s["commons_title"].replace("File:", ""), s["licence"],
s["author"][:38] for s in sources],
"Provenance of every source image. None of these files is machine "
"generated imagery.",
widths=[4.0 * cm, 6.2 * cm, 2.6 * cm, 4.2 * cm], fontsize=6.6)

h2("2.3 Construction of the test scenes")
p("Each test scene was produced by transforming the real emblem and "
"compositing it into a real photograph under controlled conditions. The "
"controlled conditions are the point of the exercise: they make it possible "
"to say precisely which factor caused a detector to fail, which is not "
"possible with an uncontrolled collection of photographs. The following "
"variations are covered.")
bullets([
"<b>Position.</b> The emblem appears at different places in the frame, "
"never at a fixed location.",
"<b>Size.</b> Scale factors from 0.55 to 1.90 relative to the template, "
"that is from roughly 53 to 182 pixels wide.",
"<b>Orientation.</b> In plane rotations of 6, 8, 15, 22, 30, 45 and 90 "
"degrees.",
"<b>Illumination.</b> Gamma correction from 0.40 (heavily under exposed) "
"to 1.9 (washed out), and a left to right or right to left brightness "
"ramp that simulates a single sided light source.",
"<b>Background.</b> Nine different real scenes, from a flat brick wall to "
"a cluttered office desk and a busy street.",
"<b>Partial occlusion.</b> From 18 to 45 per cent of the emblem is covered "
"by a textured patch copied from elsewhere in the same photograph, so the "
"occluder is not trivially separable.",
"<b>Clutter and hard negatives.</b> Scenes that contain two other "
"university emblems, and two scenes that contain no target at all, which "
"test the false positive behaviour directly.",
"<b>Sensor effects.</b> Additive Gaussian noise with a standard deviation "
"of 8 to 18 grey levels, and directional motion blur.",
])
figure("fig02_dataset.png",
"Nine of the annotated scenes, one per condition, with the ground truth "
"boxes drawn in green. The bottom right scene contains no target and "
"only two similar emblems from other faculties.")

h2("2.4 Ground truth, splits and protocol")
p("The dataset is split into a development set of %d images containing %d "
"object instances, and a held out test set of %d images containing %d "
"instances. Every threshold and every parameter reported in Section 5 was "
"chosen on the development set only. The held out set was run once at the "
"end with the parameters already fixed, which is what Section 5.8 reports. "
"This separation is what makes the reported test numbers meaningful.")
% (ds["dev_images"], ds["dev_objects"], ds["test_images"], ds["test_objects"])
p("Bounding box annotation was produced from the known placement geometry of "
"each emblem: the box is the tight rectangle around the pixels whose alpha "
"value is non zero after the geometric transformation, so it is exact to "
"the pixel rather than approximate. Every generated scene was then inspected "
"with its box drawn on it to confirm the annotation, and Figure 2 is part of "
"that check. The annotations are stored as JSON with the condition label, "
"the scale, the rotation and the occluded fraction of each instance, which "
"is what allows the per condition analysis in Section 5.8.")

# ----- 3 methodology -----
h1("3. Methodology and justification of the algorithms")
h2("3.1 Overview")
p("A classical detector is made of three separable decisions: a <b>search "
"strategy</b> that says which image regions to examine, a <b>comparison "
"function</b> that scores how much a region looks like the object, and a "
"<b>decision rule</b> that turns scores into detections. This project keeps "
"those three parts explicit and varies them one at a time, so that each "
"experiment isolates one design choice.")
figure("fig06_pipeline_stages.png",
"The pipeline on one scene: the input, its grey scale form, the "
"correlation surface produced by the masked template comparison, the "
"part of that surface above the decision threshold, the candidate boxes "
"before non maximum suppression, and the final localisation.")

h2("3.2 Preprocessing")
p("Images are converted to grey scale before matching. Correlation is defined "
"on a single channel, and the emblem is discriminated mainly by its shape "
"and internal contrast rather than by absolute colour, which changes with ")

```



```

    "That instability is visible at low thresholds in Table 3 and is one of the "
    "reasons the useful operating region is above 0.55."
    % (num(r_e2[0][4]), num(r_e2[1][4]), num(r_e2[0][3]), num(r_e2[1][3]),
    100 * (1 - summary["template"]["mask_coverage"]),
    num(float(r_e2[0][5]) / max(1e-6, float(r_e2[1][5])), 1)))

h2("5.3 Size of the scale pyramid")
table(h_e3, [[r[0], r[1], num(r[2], 3), num(r[3]), num(r[4]), num(r[5]), num(r[6])]]
    for r in r_e3,
    "Effect of the scale pyramid on the development set. Each configuration "
    "is reported at its own best threshold.",
    widths=[4.4 * cm, 2.2 * cm, 2.0 * cm, 1.9 * cm, 1.7 * cm, 1.6 * cm, 2.2 * cm],
    fontsize=7.0)
p("A single template at the reference size reaches an F1 of %s: it finds the "
    "instances near its own scale and misses everything else, which is the "
    "clearest possible demonstration that correlation is not scale invariant. "
    "Extending the pyramid to eleven scales between 0.50 and 2.02 raises F1 to "
    "%s, at %s times the run time. The "
    "intermediate five scale pyramid is interesting: it is not simply half way, "
    "because its coarse spacing means some objects fall between two sampled "
    "scales and match neither well, and it also loses precision, which is why "
    "its F1 (%s) is no better than the single scale configuration. Scale "
    "sampling must be fine enough that every object is close to some sampled "
    "scale; a geometric ratio of about 1.15 achieves that here."
    % (num(r_e3[0][5]), num(r_e3[2][5]), num(r_e3[0][4]), num(r_e3[2][4]),
    num(float(r_e3[2][6]) / max(1e-6, float(r_e3[0][6])), 1), num(r_e3[1][5])))

h2("5.4 Rotated template bank")
table(h_e4, [[r[0], r[1], num(r[2], 3), num(r[3]), num(r[4]), num(r[5]), num(r[6])]]
    for r in r_e4,
    "Adding eight rotated copies of every template. Recall is reported "
    "separately on the rotated scenes and on the viewpoint scenes.",
    widths=[3.6 * cm, 2.2 * cm, 2.0 * cm, 1.8 * cm, 2.6 * cm, 2.6 * cm, 2.2 * cm],
    fontsize=7.0)
p("Without rotated templates the detector finds none of the rotated instances "
    "at all: recall on the rotation scenes is %s. Adding eight angles raises "
    "that to %s, but multiplies the bank from %s to %s templates and the run "
    "time from %s to %s seconds per image, and overall F1 does not improve "
    "(%s against %s) because the enlarged bank also produces more opportunities "
    "for a background window to score highly. This is the honest result: buying "
    "rotation invariance by brute force is expensive and only partly effective, "
    "since a rotation that falls between two sampled angles still matches "
    "poorly. It is the strongest argument in this project for the feature based "
    "approach of Section 5.6, which is rotation invariant by construction and "
    "costs less."
    % (num(r_e4[0][4]), num(r_e4[1][4]), r_e4[0][1], r_e4[1][1],
    num(r_e4[0][6]), num(r_e4[1][6]), num(r_e4[0][3]), num(r_e4[1][3])))

h2("5.5 The explicit sliding window detector")
p("This experiment measures the sliding window detector directly, varying the "
    "set of window sizes and the step size, and recording both the number of "
    "windows evaluated and the time taken. It was run on a five image subset of "
    "the development set that includes a scene with two objects and a scene "
    "with none.")
table(h_e5, [[r[0], r[1], r[2], r[3], num(r[4]), num(r[5], 2), r[6], r[7], r[8],
    num(r[9]), num(r[10]), num(r[11])]] for r in r_e5,
    "Sliding window detector: window sizes, step size, cost and accuracy. "
    "Each row is reported at its own best threshold.",
    widths=[2.9 * cm, 1.3 * cm, 1.0 * cm, 1.9 * cm, 1.6 * cm, 1.3 * cm,
    1.0 * cm, 1.0 * cm, 1.0 * cm, 1.3 * cm, 1.1 * cm, 1.0 * cm],
    fontsize=6.2)
figure("fig05_sliding_window.png",
    "Left, seconds per image against step size on a logarithmic axis. "
    "Right, F1 against step size. The cost falls as the square of the step "
    "while accuracy collapses beyond a step of about eight pixels.")
p("The relationship between step size, detection performance and execution "
    "time is the clearest quantitative result in this project. With three "
    "window sizes, a step of 2 pixels evaluates %s windows and costs %s seconds "
    "per image, while a step of 24 pixels evaluates %s windows and costs %s "
    "seconds, a saving of about %s times, exactly as predicted by the inverse "
    "square relationship between step and window count. Accuracy moves the "
    "other way: F1 falls from %s at a step of 2 to %s at a step of 24. There "
    "are two distinct reasons for that fall, and they are worth separating. "
    "The first is a scoring effect: a window that is offset from the object by "
    "half a step contains part of the background and its correlation drops, so "
    "the object can fall below the threshold entirely. The second is a "
    "localisation effect: even when the object is detected, a coarse grid "
    "cannot place the box within the 0.5 intersection over union required to "
    "count as a true positive. A step of about one twentieth of the window "
    "size, which is 4 to 8 pixels here, is the practical compromise, and it is "
    "the configuration carried into Section 5.8. The fall is not perfectly "
    "monotonic: a step of 16 finds nothing at all while a step of 24 finds one "
    "object. That is not noise in the measurement, it is grid alignment. "
    "Whether a coarse grid happens to place a window close enough to an object "
    "depends on the object position modulo the step, so a coarse detector "
    "succeeds or fails almost by luck, which is itself a good reason not to use "
    "one."
    % (sw_fine[3], num(sw_fine[4], 2), sw_coarse[3], num(sw_coarse[4], 3),
    num(float(sw_fine[4]) / max(1e-6, float(sw_coarse[4])), 0),
    num(sw_fine[11]), num(sw_coarse[11])))
table(h_e5b, [[num(r[0], 2)] + r[1:4] + [num(r[4]), num(r[5]), num(r[6])]]
    for r in r_e5b,
    "Threshold sweep for the explicit sliding window detector at a fixed "
    "geometry (three window sizes, step 4).",
    widths=[2.4 * cm] + [2.1 * cm] * 6, fontsize=7.0)

h2("5.6 SIFT feature matching with RANSAC")
p("Two parameters govern the feature based detector: Lowe's ratio, which "
    "decides how distinctive a correspondence must be to survive, and the "
    "minimum number of RANSAC inliers required before an instance is accepted, ")

```

```

    "which is the detection decision for this method.")
table(h_e6, [[num(r[0], 2), r[1], r[2], r[3], r[4], num(r[5]), num(r[6]),
    num(r[7]), num(r[8])] for r in r_e6],
    "Ratio test and inlier threshold study on the development set.",
    widths=[1.9 * cm, 1.9 * cm, 1.9 * cm, 2.0 * cm, 2.2 * cm, 1.9 * cm,
    1.7 * cm, 1.5 * cm, 2.0 * cm], fontsize=6.8)
figure("fig09_sift_matches.png",
    "Correspondences that survive the ratio test between the template and "
    "a scene in which the emblem is rotated by 30 degrees. Correlation "
    "based matching fails completely on this image unless a rotated "
    "template bank is used; feature matching handles it without any "
    "additional search.")
p("The best configuration on the development set is a ratio of %s with %s "
    "inliers required, giving an F1 of %s at %s seconds per image, which is "
    "faster than the eleven scale correlation search and far faster than the "
    "rotated bank. The inlier count behaves exactly like the correlation "
    "threshold: a low requirement admits spurious homographies fitted to "
    "accidental matches on the distractor emblems, and a high requirement "
    "rejects the small and the heavily occluded instances, which simply do not "
    "have enough keypoints to reach the count. The method has one structural "
    "weakness that the correlation detector does not have: it needs texture. "
    "The emblem at 0.55 scale is only about 53 pixels wide, and SIFT finds few "
    "stable keypoints at that size."
    % (num(e6_best[0], 2), e6_best[1], num(e6_best[7]), num(e6_best[8])))

h2("5.7 Colour proposals combined with template matching")
table(h_e7, [[r[0], num(r[1]), num(r[2], 4), num(r[3], 3), num(r[4]), num(r[5]),
    num(r[6])] for r in r_e7],
    "Restricting the correlation search to colour proposals rather than "
    "searching the whole image.",
    widths=[5.0 * cm, 2.2 * cm, 2.6 * cm, 1.9 * cm, 1.7 * cm, 1.5 * cm, 1.5 * cm],
    fontsize=7.0)
p("The proposal stage reduces the searched area to %s per cent of the image "
    "and the run time from %s to %s seconds per image, while F1 changes from %s "
    "to %s. The gain is worth understanding precisely, because it does not come "
    "from a better comparison function, which is unchanged. Removing seventy "
    "per cent of the image removes seventy per cent of the opportunities for a "
    "background window to score highly, so the detector can be operated at a "
    "lower threshold (%s rather than %s) without paying for it in false "
    "positives, and the lower threshold is what recovers the extra objects: "
    "recall rises from %s to %s at unchanged precision. But the proposal stage "
    "is itself a detector, and "
    "when its colour threshold rejects an emblem, for example in the heavily "
    "under exposed scene where the navy shield is nearly black and its "
    "saturation collapses, the correlation stage never gets the chance to look "
    "there. A cascade is only ever as good as its first stage, which is a "
    "general property of cascaded detectors and not a peculiarity of this "
    "implementation."
    % (num(100 * float(r_e7[1][2]), 1), num(r_e7[0][1]), num(r_e7[1][1]),
    num(r_e7[0][6]), num(r_e7[1][6]), num(r_e7[1][3], 3), num(r_e7[0][3], 3),
    num(r_e7[0][5]), num(r_e7[1][5])))

h2("5.8 Held out test set: final comparison")
p("Every method was then run once on the %d held out images, with all "
    "parameters fixed at the values selected on the development set. No "
    "parameter was adjusted after seeing these numbers." % ds["test_images"])
table(h_e8, [[r[0], r[1], r[2], r[3],
    r[4], num(r[5]), num(r[6]), num(r[7]), num(r[8]), num(r[9])]
    for r in r_e8],
    "Final comparison on the held out test set, at IoU 0.5. The operating "
    "point is a correlation threshold for the correlation based methods and "
    "a minimum RANSAC inlier count for the feature based method.",
    widths=[4.0 * cm, 1.6 * cm, 1.2 * cm, 1.2 * cm, 1.3 * cm, 1.4 * cm,
    1.3 * cm, 1.1 * cm, 1.5 * cm, 1.4 * cm], fontsize=6.6)
table(h_e8b, r_e8b, "F1 by imaging condition on the held out test set. The "
    "negative images are excluded from this table because F1 is undefined "
    "when there is no object to find; their contribution appears as false "
    "positives in the previous table.",
    widths=None, fontsize=6.4)
figure("fig08_per_condition.png",
    "F1 by imaging condition on the held out test set. The conditions on "
    "which correlation collapses are exactly the ones it has no invariance "
    "to.")
p("The best method on the held out set is &lt;b>%s&lt;/b>, with a precision of %s, "
    "a recall of %s and an F1 of %s. Localisation is good wherever detection "
    "succeeds: the mean intersection over union of the true positives is %s, "
    "well above the 0.5 acceptance threshold, which says that the failures in "
    "this system are failures to detect rather than failures to localise."
    % (e8_best[0], num(e8_best[5]), num(e8_best[6]), num(e8_best[7]),
    num(e8_best[8])))
p("The ordering of the methods on the held out set repeats the ordering on "
    "the development set, which is the first thing to check, and the per "
    "condition table explains it. The correlation detectors score 1.0 on the "
    "clutter scenes and at or near 1.0 on the baseline scenes, and 0.0 on "
    "rotation, viewpoint, occlusion and noise. Feature matching scores 1.0 on "
    "rotation and is the only single method that scores above zero on noise and "
    "on occlusion, which is why combining the two recovers almost everything: "
    "the two families fail on disjoint conditions. Two results deserve to be "
    "stated plainly rather than smoothed over. First, no method in this study "
    "detected the viewpoint scene: a projective tilt of twenty per cent of the "
    "object width defeated the correlation detectors outright, and on the "
    "feature side only eleven correspondences survived the ratio test, fewer "
    "than the %s that the operating point demands, so no homography was even "
    "fitted. Second, the explicit sliding window detector scores worst of all "
    "on the test set while costing %s seconds per image, because a step of 4 "
    "pixels combined with only three window sizes samples scale far too "
    "coarsely; the dense one pixel search of cv2.matchTemplate over eleven "
    "scales is both cheaper and better, which is the practical argument for "
    "using the optimised correlation rather than an explicit loop once the "
    "algorithm is understood."

```

```

% (e6_best[1], num(sw_row[9], 1)))
figure("fig10_success.png",
    "Correct detections by multi scale template matching on the held out "
    "set. Yellow is ground truth, green is the detection.")
figure("fig11_failure.png",
    "Failure cases for multi scale template matching on the held out set. "
    "Yellow is ground truth, green is the detection. Rotation, viewpoint "
    "change and heavy occlusion account for almost every failure.")

# ----- 6 discussion -----
h1("6. Critical discussion: when these methods become unreliable")
p("The practical question for any deployment is when template matching and "
    "sliding window detection become unreliable. The per condition results in "
    "Table 12 answer "
    "that empirically, and the reason in each case follows from what the "
    "correlation score actually computes: a comparison of pixel values at fixed "
    "relative positions inside a rectangle.")

h3("Scale variation")
p("A template of a fixed size compared with an object of a different size "
    "compares the wrong pixels with each other. A scale error of 20 per cent is "
    "already enough to push a correct match below a threshold that is tight "
    "enough to control false positives, which is why the single scale detector "
    "in Table 5 reaches a recall of only 5%. The pyramid solves this at linear "
    "cost in the number of scales, but only if the sampling is fine enough that "
    "every object lies close to some sampled scale. Scale is the best behaved "
    "of the failure modes because it is one dimensional and cheap to enumerate."
    % num(r_e3[0][4]))

h3("Rotation")
p("Rotation is worse than scale for two reasons. First, the search is over an "
    "additional dimension, so a bank that covers rotation as finely as it covers "
    "scale is an order of magnitude larger. Second, rotating a rectangular "
    "template introduces corners that contain no object, which interacts badly "
    "with the correlation unless the mask is rotated with the template. Table 6 "
    "shows the consequence: without rotated templates, recall on the rotated "
    "scenes is zero, and even with eight angles it only reaches 5% while "
    "costing 5s seconds per image. Feature matching is the right answer here, "
    "because SIFT descriptors are computed relative to a dominant gradient "
    "orientation and are therefore rotation invariant without any search at all."
    % (num(r_e4[1][4]), num(r_e4[1][6]))))

h3("Occlusion")
p("Correlation is a sum over all pixels of the template, so an occluder that "
    "covers a fraction f of the object corrupts a fraction f of the terms in "
    "that sum, and the score falls roughly in proportion. Because the "
    "normalisation is computed over the whole mask, an occluded region does not "
    "merely contribute nothing, it contributes actively wrong values. In this "
    "study the correlation detector missed both occluded instances in the held "
    "out set at its selected operating point, including the one with only 20 "
    "per cent of the emblem covered. Feature matching degrades far more "
    "gracefully: it recovered the lightly occluded instance and failed only on "
    "the heavily occluded one, where 45 per cent of the emblem including the "
    "scroll and the lower shield is hidden. It needs only enough visible "
    "keypoints to fit a homography, and RANSAC discards the rest as outliers.")

h3("Illumination")
p("This is the one condition that the chosen comparison function genuinely "
    "handles. Because the mean is subtracted and the result is divided by the "
    "norms, an affine change of intensity leaves the score unchanged, and the "
    "detector survives the one sided brightness ramp and the milder gamma "
    "changes in this dataset. It fails when the illumination change is not "
    "affine: strong gamma compression is a non linear map that changes the "
    "internal contrast ratios of the emblem, and at a gamma of 0.40 the "
    "correlation detector missed the held out scene outright, because the dark "
    "shield is crushed towards black until quantisation and JPEG noise destroy "
    "its internal structure. Feature matching still found that instance. A one "
    "sided brightness ramp is closer to affine over a region as small as the "
    "emblem, and the detector survives it.")

h3("Sensor noise and blur")
p("Additive noise with a standard deviation of 18 grey levels was enough to "
    "push the correlation score of a correct match below the operating "
    "threshold, giving an F1 of zero on the noise condition for every "
    "correlation based method in Table 12. The mechanism is that noise inflates "
    "the norm of the window without contributing anything correlated with the "
    "template, so the normalised score is scaled down. Smoothing the image "
    "before matching recovers part of this, at the cost of blurring the fine "
    "internal structure of the emblem that the match depends on, which is why "
    "the preprocessing function exposes the smoothing radius rather than fixing "
    "it. Feature matching still found the emblem in the noisy scene, although it "
    "also produced a false positive there, because SIFT smooths at each octave "
    "and its descriptors pool gradients over a region rather than comparing "
    "pixel values one by one.")

h3("Background clutter and the number of windows examined")
p("Precision in this problem is dominated by how many windows are examined. A "
    "960 by 640 image contains roughly half a million window positions per "
    "scale, so even a false positive rate of one in a hundred thousand produces "
    "several false detections per image, and multiplying the template bank "
    "multiplies that number. This is why Table 3 shows more than two thousand "
    "false detections at a threshold of 0.30 and none at 5%, and why the colour "
    "proposal stage of Section 5.7, which removes about 5% per cent of the "
    "search space, helps precision without touching the comparison function. "
    "Clutter that resembles the target in colour and layout, such as the other "
    "university emblems used here, is the hardest case: it survives the colour "
    "proposal stage and can only be rejected by the comparison function itself."
    % (num(tm_best[0], 3), num(100 * (1 - float(r_e7[1][2])), 0)))

h3("Viewpoint change")

```

```

p("A change of viewpoint applies a projective transformation to a planar "
  "object. Correlation has no model for that at all: it cannot be enumerated "
  "cheaply, because the space of homographies is eight dimensional, and a "
  "template bank over that space is not feasible. This is where the classical "
  "correlation detector genuinely runs out of road, and where feature "
  "matching with RANSAC is not merely faster but structurally correct, since "
  "the homography it estimates is exactly the transformation that occurred. "
  "It should be said that being structurally correct was not enough here: on "
  "the held out viewpoint scene the feature detector found correspondences on "
  "the emblem but fell short of the inlier count required by the operating "
  "point, so it reported nothing. The tilt reduces the effective resolution "
  "of the emblem along one axis and destroys a proportion of the keypoints, "
  "and a threshold tuned on mostly frontal development images is then too "
  "strict. Viewpoint change was the single condition that no method in this "
  "study handled.")

h2("6.1 Limitations of this study")
bullets([
  "The evaluation set is modest, %d images with %d instances in total. The "
  "differences between the stronger methods in Table 11 rest on a small "
  "number of instances and should be read as indicative rather than "
  "definitive.",
  % (ds["dev_images"] + ds["test_images"], ds["dev_objects"] + ds["test_objects"]),
  "The scenes are real photographs with a real emblem composited into them "
  "under controlled transformations. This gives exact ground truth and "
  "isolates each factor cleanly, but it does not reproduce every effect of "
  "photographing a real printed emblem, in particular specular highlights "
  "on glossy signage, surface curvature and lens distortion.",
  "The colour proposal stage is tuned to the navy blue of this particular "
  "emblem and would need to be retuned for another object, whereas the "
  "correlation and feature stages are generic.",
  "Only in plane rotation was tested at a limited set of angles, and the "
  "viewpoint changes are moderate tilts rather than extreme oblique views.",
  "Timings are single threaded Python and OpenCV on one machine. The "
  "relative costs are meaningful, the absolute numbers are not portable.",
])

h2("6.2 How the system could be improved")
bullets([
  "Use edge or gradient orientation information instead of raw intensity, "
  "for example the chamfer distance on Canny edges or a histogram of "
  "oriented gradients descriptor, both of which are far less sensitive to "
  "photometric change than raw pixel correlation.",
  "Build a proper cascade: a very cheap rejector on integral image features "
  "first, then correlation, then feature verification, so that the "
  "expensive comparison only ever sees a handful of windows.",
  "Verify every correlation detection with the feature matcher and keep "
  "only detections that both stages agree on, which trades a little recall "
  "for high precision, or keep detections that either stage finds, which "
  "does the reverse. The combined method in Table 11 is a simple version of "
  "the second option.",
  "Estimate a dominant orientation for each candidate region from its "
  "gradient histogram and rotate the window to a canonical orientation "
  "before correlating, which would give most of the benefit of the rotated "
  "bank at a small fraction of its cost.",
])

# ----- 7 conclusion -----
h1("7. Conclusion")
p("A complete classical detection and localisation system for the University "
  "of Ghana coat of arms was built, and the full progression from search "
  "strategy through template comparison and detection decision to object "
  "localisation and quantitative evaluation was implemented and measured. On "
  "a held out set of %d images that was never used for tuning, the best "
  "configuration reached a precision of %s, a recall of %s and an F1 of %s at "
  "an intersection over union threshold of 0.5, with a mean intersection over "
  "union of %s on its correct detections.",
  % (ds["test_images"], num(e8_best[5]), num(e8_best[6]), num(e8_best[7]),
    num(e8_best[8])))
p("The experiments support four conclusions. First, the decision threshold is "
  "the single most powerful parameter in a correlation detector, and the "
  "transition from thousands of false detections to none spans a range of "
  "about 0.05 in correlation, which means the threshold must be selected on "
  "data and not guessed. Second, invariance in a correlation detector has to "
  "be paid for by enumeration: scale costs a linear factor and is worth "
  "paying, rotation costs an order of magnitude and is not, and viewpoint "
  "cannot be enumerated at all. Third, the cost of a sliding window search "
  "falls as the square of the step size while its accuracy collapses once the "
  "step exceeds roughly one twentieth of the window, which makes a step of 4 "
  "to 8 pixels the practical compromise here. Fourth, the classical methods "
  "are complementary rather than competing: colour segmentation is the "
  "cheapest way to cut the search space, correlation is the most precise "
  "comparison when the object appears roughly upright and unoccluded, and "
  "SIFT with RANSAC is the only one of the three that survives rotation, "
  "sensor noise and partial occlusion. A system that uses all three, in that "
  "order, is better than any of them alone. Viewpoint change defeated every "
  "method tested and remains the open problem in this pipeline.")

# ----- references -----
h1("References")
for i, ref in enumerate([
  "Lewis, J. P. (1995). Fast normalized cross correlation. Vision Interface, "
  "pages 120 to 123.",
  "Lowe, D. G. (2004). Distinctive image features from scale invariant "
  "keypoints. International Journal of Computer Vision, 60(2), 91 to 110.",
  "Fischler, M. A. and Bolles, R. C. (1981). Random sample consensus: a "
  "paradigm for model fitting with applications to image analysis and "
  "automated cartography. Communications of the ACM, 24(6), 381 to 395.",
  "Viola, P. and Jones, M. (2001). Rapid object detection using a boosted "
  "cascade of simple features. Proceedings of CVPR, pages 511 to 518.",
])

```

```

"Dalal, N. and Triggs, B. (2005). Histograms of oriented gradients for "
"human detection. Proceedings of CVPR, pages 886 to 893.",
"Szeliski, R. (2022). Computer Vision: Algorithms and Applications, "
"second edition. Springer.",
"Bradski, G. (2000). The OpenCV Library. Dr. Dobb's Journal of Software "
"Tools.",
"Everingham, M., Van Gool, L., Williams, C. K. I., Winn, J. and "
"Zisserman, A. (2010). The PASCAL Visual Object Classes challenge. "
"International Journal of Computer Vision, 88(2), 303 to 338.",
], 1):
    p("[%d] %s" % (i, ref), ParagraphStyle("ref", parent=BODY, leftIndent=16,
                                           firstLineIndent=-16, spaceAfter=4))

# ----- appendix -----
story.append(PageBreak())
h1("Appendix A. Complete source code")
p("The listing below is the complete source of the system, in the order in "
"which the files are run. The same files are included in the repository.")
for fname in ["download_data.py", "build_dataset.py", "detector.py",
              "evaluate.py", "run_experiments.py", "make_report.py"]:
    path = os.path.join(ROOT, "src", fname)
    if not os.path.exists(path):
        continue
    h2("src/" + fname)
    with open(path, "r", encoding="utf-8") as fh:
        lines = fh.read().split("\n")
    chunk = []
    for line in lines:
        chunk.append(esc(line.rstrip()[:112]))
        if len(chunk) == 46:
            story.append(Preformatted("\n".join(chunk), CODE))
            chunk = []
    if chunk:
        story.append(Preformatted("\n".join(chunk), CODE))
    story.append(Spacer(1, 6))

os.makedirs(os.path.dirname(OUT), exist_ok=True)
build()
print("wrote", OUT)

if __name__ == "__main__":
    main()

```